CSE 3200: System Development Project

MEDORA: AN AI-NATIVE HEALTHCARE PLATFORM FOR BANGLADESH

By

Sarwad Hasan Siddiqui
Roll: 2107006

&

Adiba Tahsin
Roll: 2107031

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh

April, 2026

MEDORA: An AI-Native Healthcare Platform for Bangladesh

By

Sarwad Hasan Siddiqui
Roll: 2107006

&

Adiba Tahsin
Roll: 2107031

Supervisor:

Kazi Saeed Alam
Assistant Professor
Department of Computer Science and Engineering
Khulna University of Engineering & Technology

Signature

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh
April, 2026

Acknowledgment

In the name of God, who gave us the ability to carry out this project successfully. Firstly, we would like to extend our heartfelt thanks to our honorable guide, Kazi Saeed Alam Sir, Assistant Professor, Department of Computer Science and Engineering, Khulna University of Engineering & Technology, for the invaluable guidance provided by him throughout this project. Likewise, our thanks go out to the esteemed faculty members of our department for their invaluable suggestions at the review sessions, the open source community whose software was used in the development of this system, and most importantly, our parents and families.

Authors

Abstract

Bangladesh's health care sector struggles to integrate appointments, telemedicine, prescriptions, and data records into a cohesive process. Medora fills this niche by developing an integrated AI-enabled health care platform that consolidates authenticated appointment booking, bi-lingual support, OCR prescription scanning, consent-aware record management, and safety-controlled AI-assistance within a single system. Medora was built by using Next.js for a progressive web application, FastAPI for the back-end service coupled with Supabase's PostgreSQL database, and a custom OCR/AI microservice with schema validation, privacy, and role-based orchestration. An evaluation found that the project accomplished its key goals, which include efficient appointment workflows, proper organization of patient data, advanced analytics and personalization for users, AI native environment, OCR processing times around 10.5 seconds, high accuracy in OCR field identification, and consistent performance under representative loads.

Keywords: Healthcare informatics, AI-assisted medicine, prescription OCR, real-time appointment scheduling, service-oriented architecture, progressive web application, consent-aware design.

Contents

List of Tables

List of Figures

CHAPTER 1

Introduction

1.1 Introduction

The digital transformation of health care is already an engineering and policy reality, especially for developing countries. For example, Bangladesh, whose population exceeds 170 million but whose number of physicians per capita is limited, stands to benefit from any incremental optimization in terms of appointment scheduling, continuity of records, decision-support system, etc. However, the state-of-the-art situation remains quite primitive: some app to make an appointment, another app for teleconsultation, a separate hospital system, and prescriptions written in paper outside of the entire ecosystem. Information flows poorly between those channels, data consent is hardly enforced, and there is little continuity between doctor visits. Medora aims to explore the possibility of creating a complete, consent-aware, and AI-augmented health care platform for an academic organization.

Medora is meant to be AI-native rather than merely AI-powered. The problem with AI bolted on to a product is that either the input lacks the required structure, or the main application does not have the means to enforce AI security measures. In AI-bolted architectures, the user feels that inconsistencies arise in terms of how the application behaves. On the contrary, Medora's AI orchestration, schema validation, data consent enforcement, and observability constitute a single architecture layer, and every time the model makes a prediction, its call goes through the backend orchestrator responsible for providing a correct provider, anonymizing PII, validating the output according to a Pydantic schema, and storing conversation turns for auditing purposes.

Before moving into the technical details, Figure 1.1 gives a plain-language picture of what Medora does for everyday users. Patients, doctors and administrators all meet inside one app that helps people find care, book visits, keep a clear health history, read paper prescriptions automatically, get timely reminders, and talk to a friendly assistant — all in English or Bangla, and even when the internet is weak.

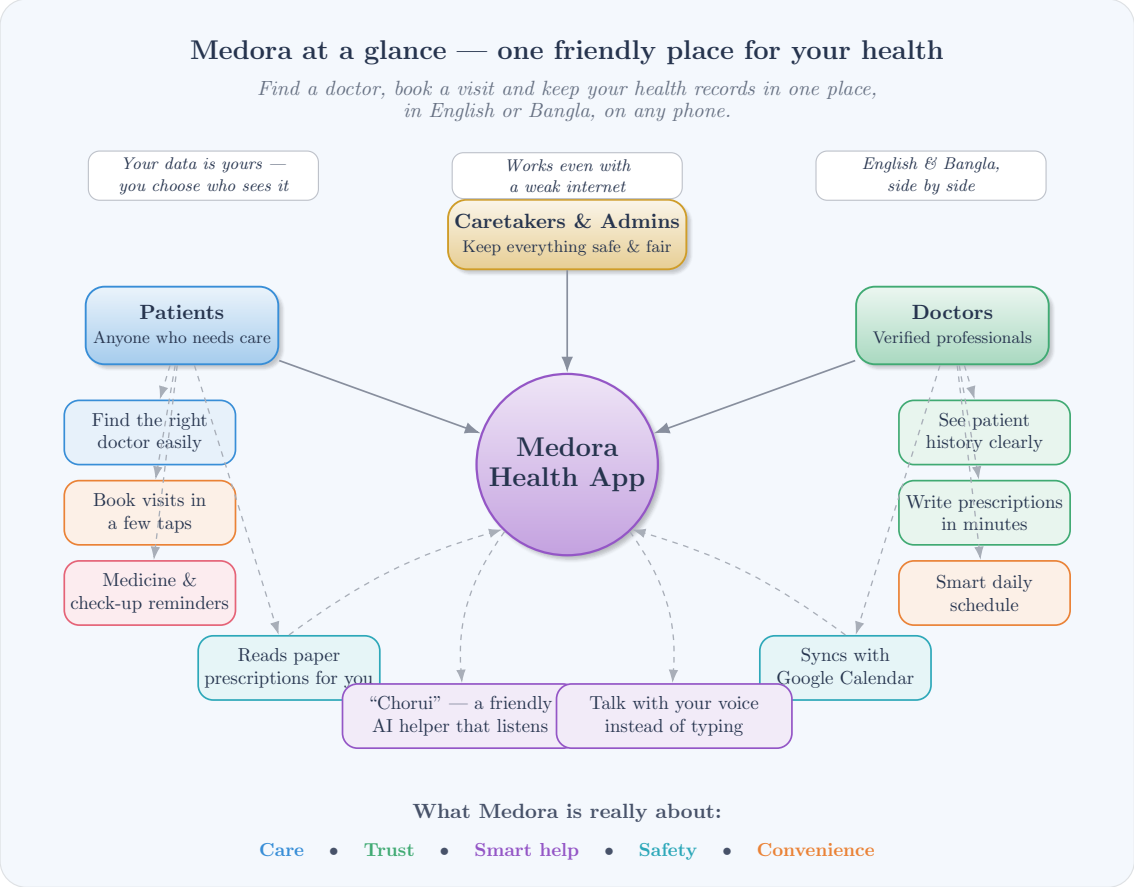


Figure 1.1: *A plain-English look at what Medora does. Patients, doctors and admins all use the same app to find doctors, book visits, keep health records, read prescriptions, set reminders and chat with an AI assistant, in English or Bangla.*

1.2 Background and Problem Statement

The Bangladeshi patient experience includes interacting with more than four systems separately. Finding a doctor involves search engines or referral systems or outdated directory listings. Booking an appointment takes place using different routes in which the lack of synchronization in updating leads to time-slot issues. Doctors either meet in person or via telemedicine solutions without maintaining longitudinal history data. Prescriptions are predominantly issued in paper format and can be challenging to comprehend for patients, pharmacists, and insurance companies. Consequently, there are many issues such as missed visits, duplicate test procedures, medication errors, and disputes due to a lack of credible proof.

Existing products address only one aspect of the patient journey. Healtha and Medexly enable doctor searches but lack booking functionalities. DocTime and Sebaghar offer telemedicine services but lack continuity of record management. Doctorola and GoodDoktor help users book appointments but have no triage system or prescription document analysis capabilities.

Epic and Cerner maintain detailed clinical histories, but such records cannot be accessed by local user interfaces. Teladoc and Babylon Health offer full-service solutions worldwide but are not compatible with the linguistic and pricing needs of Bangladesh. This makes it evident that the main research gap lies in the absence of a platform that covers all these aspects—structured onboarding, verified doctors, scheduling, prescription analysis, and safe AI triaging—under a Bangla and English interface.

1.3 Objectives

Project objectives were clearly identified in advance, and stated in concrete terms that could be measured against actual results. We formulated five main objectives and monitored their progress throughout our development process.

Our first objective was to create a single platform solution featuring role-based workflows for patients, doctors, and administrators, with access restrictions enforced at all levels of architecture. Our second objective was to implement AI-powered capabilities in our app such as language-aware physician search and Chorui's conversational interface, but prevent any form of autonomous diagnostic and treatment procedures via guardrails. Third, we aimed to create a prescription OCR pipeline that can accurately digitize handwritten prescription slips and medical documents in both English and Bangla. Fourth, we ensured seamless real-time synchronization of appointment states between clients to avoid double booking and scheduling conflicts. Finally, the fifth objective was to deploy the solution in production using public cloud services and CI/CD pipeline with observability and performance budget management.

Five secondary objectives were also identified: supporting two languages (English and Bangla), progressive web application capabilities, including offline functionality, mobile-first approach starting at a 320px screen resolution, comprehensive test coverage, including unit, integration, and end-to-end testing, and adequate documentation allowing an external reviewer to validate our architecture.

1.4 Scope

The scope covers the full stack needed to meet the objectives. On the client side, we built a Next.js 16 application on React 19 and TypeScript 5, packaged as a progressive web application through Serwist and styled with Tailwind CSS 4 plus shadcn/ui on top of Radix UI primitives. Supabase handles authentication and real-time channels. Internationalisation is implemented with next-intl and English/Bangla catalogs across ten functional categories.

On the server side, a FastAPI service on Python 3.11 and Uvicorn exposes 194 endpoint

handlers across 26 route modules. Data persistence uses asynchronous SQLAlchemy 2 on Supabase PostgreSQL, and schema evolution is managed through 47 Alembic migrations. Security combines Supabase JWT verification, role-based access control through FastAPI dependencies, and a consent guard for sensitive patient-health queries. Voice input uses faster-whisper for speech recognition, and push notifications are sent through a VAPID channel with pywebpush.

The AI layer has two coordinated parts. One is a provider-agnostic orchestration service for Groq, Gemini and Cerebras models with schema-validated output. The other is a dedicated OCR microservice that uses YOLO ONNX for region detection, Azure Document Intelligence as the primary OCR engine, PaddleOCR as fallback, and RapidFuzz for medicine-name resolution against a curated catalog. Deployment runs on Microsoft Azure Container Apps with GitHub Actions pipelines for backend, AI OCR and performance-budget checks.

1.5 Novelty of the Approach

The novelty of Medora does not derive from the creation of a novel algorithm but the way in which well-known technologies have been combined in a manner that we do not observe in either the literature surveyed or currently on the market. Three components in particular were novel in their composition. Firstly, the platform integrates structured data collection of patient history, real-time booking of appointments and prescription paper intelligence into one consent-enabled platform optimised for a bilingual context in Bangladesh. Secondly, the pathway of AI orchestration has been designed as safety bounded, where every interaction with an API includes anonymisation, schema validation and navigation through confidence, with deterministic backend logic retaining the final say in decisions regarding patients. Thirdly, the platform has been deployed to production standards within the time-frame of an academic project including container orchestration, observability dashboards and performance budgets ensured by CI. Together, these show that clinical safety, software engineering quality and operational readiness can be achieved in one academic deliverable through disciplined architecture.

1.6 Project Planning and Work Distribution

We used an agile software development lifecycle in our project lifecycle planning, where there are multiple iteration cycles of requirements, design, implementation, testing, deployment and feedback, as shown in Fig. 1.2. We divided our project into four work phases over three months (Jan 2026 - April 2026) with specific deliverables at the end of each phase. These phases are as follows:

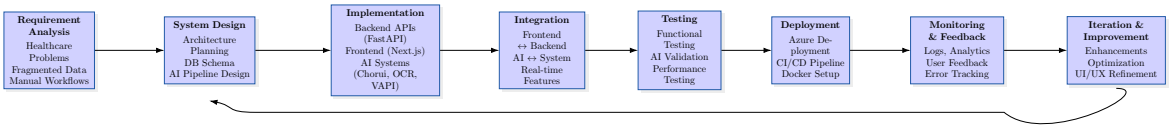


Figure 1.2: *The development cycle we used: plan, build, test, release and repeat. Each time, we fed that back into the process.*

1. Foundation, where we did the authentication, role-based user on-boarding, appointment booking and AI-based doctor search;

2. Features, where multi-type prescription, notifications, docker containerization and reminder loops were implemented;

3. AI and polish, where we implemented an OCR microservice, Chorui assistant, calendar OAuth and progressive web app packaging; and

4. More advanced features, where the re-scheduling of notifications, analytics dashboard, voice control, bilingual catalog and observability stacks were added.

Figure 1.3 shows the complete development lifecycle journey across these stages, and the evolution of the system from the beginning to deployment.

Task allocation was done in a feature branching and merge git workflow with 15 explicit merge commits at integration points. The collaborators were kept in-sync through daily meetings and a task list. In parallel areas such as the prescription module where the backend and frontend were both being developed, an ad hoc Responsible, Accountable, Consulted, Informed (RACI) approach was adopted to quickly resolve questions of ownership. Figure 1.4 captures the workflow of collaboration between task planning, parallel work, review and integration.

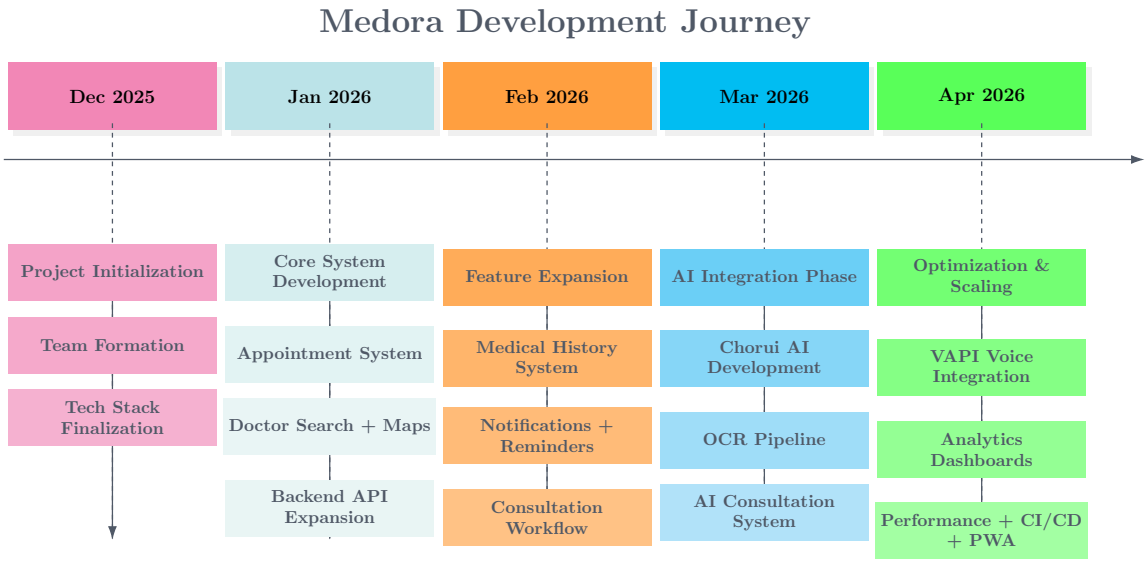


Figure 1.3: *How the project evolved: starting from the core platform, adding features phase by phase, bringing in AI, and finishing with the production-readiness work.*

Table 1.1: *Who built what: a breakdown of each workstream between the two of us.*

Workstream	**Sarwad Hasan Siddiqui (2107006)**	**Adiba Tahsin (2107031)**
Backend architecture	Lead (FastAPI, async SQLAlchemy, 194 endpoints, 41 models, 47 migrations)	Schema review and consumer-side validation
Frontend architecture	Server-action contracts, security hooks	Lead (Next.js 16, App Router, 45 components)
AI orchestration	Lead (Groq/Gemini/Cerebras, schema validation, PII anonymisation)	Chorui chat UI, voice control, navigation policy
OCR microservice	Lead (YOLO ONNX, Azure OCR, parsers, medicine matcher)	UX for upload, review and correction flows
DevOps and deployment	Lead (Dockerfiles, Azure Container Apps, GitHub Actions, Grafana)	Lighthouse CI integration and dashboard authoring
Internationalisation	Backend message catalogue scaffolding	Lead (next-intl, English and Bangla, 10 categories)
Testing	Backend unit, integration, security, load tests	Playwright end-to-end and frontend performance tests
Documentation	Backend PRD, deployment guide	Frontend PRD, Chorui pipeline, PWA documentation

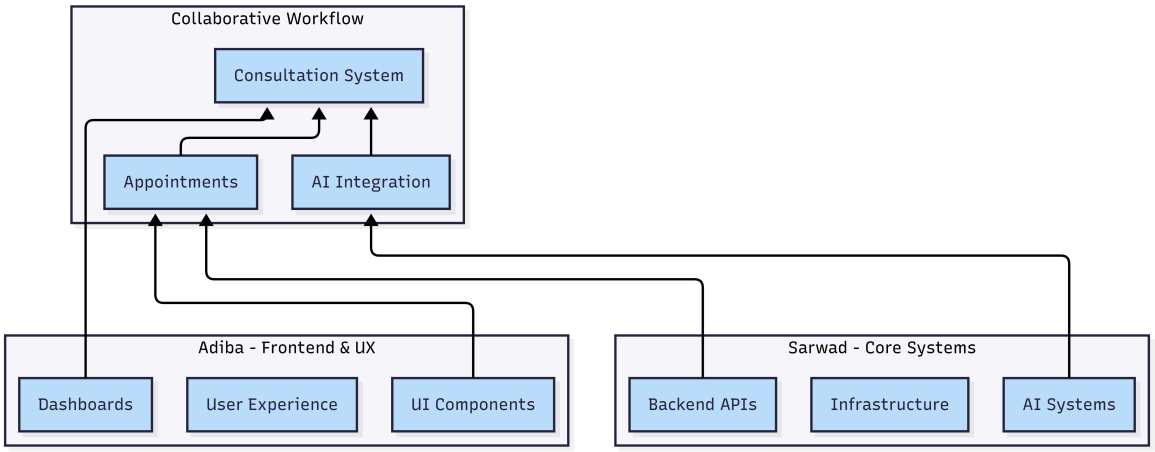


Figure 1.4: *How the two of us worked together: planning together, splitting off to build in parallel, reviewing each other's work and merging at set checkpoints.*

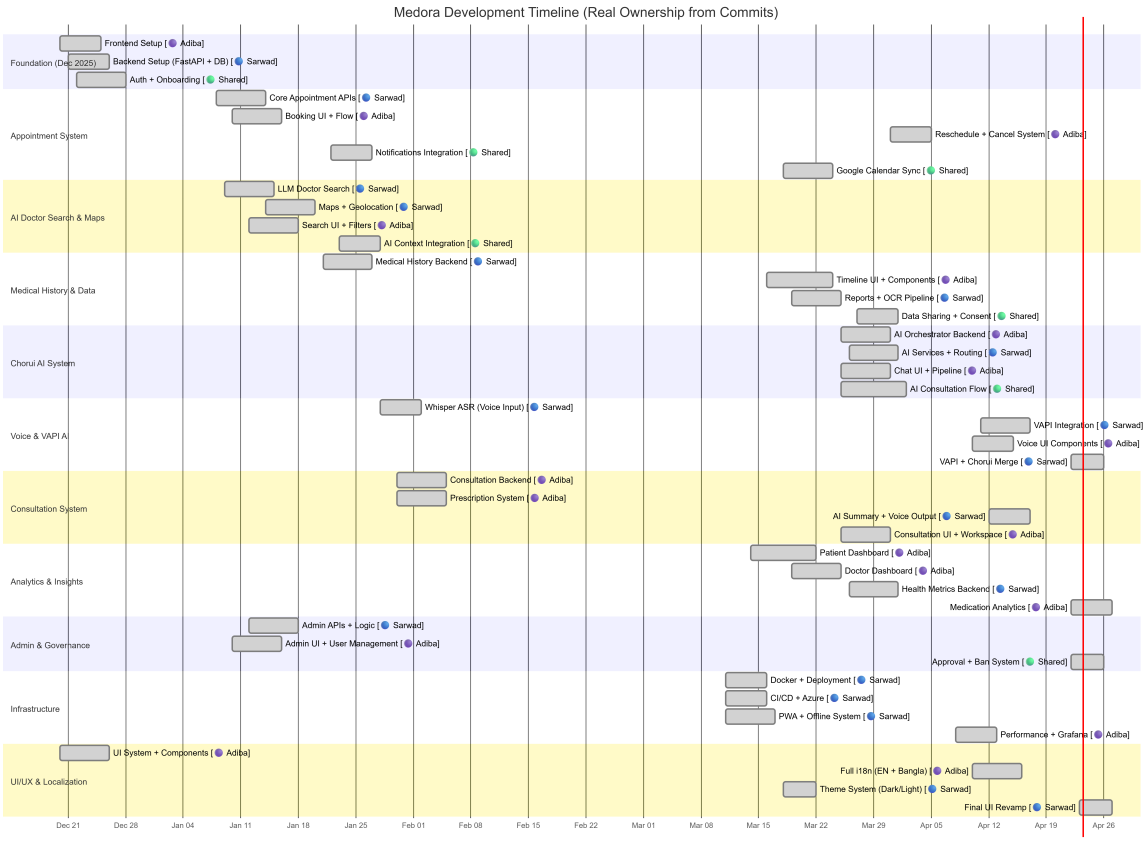


Figure 1.5: *Timeline of the four build phases, December 2025 to April 2026.*

1.7 Applications of the Work

Medora has practical value for all three stakeholder groups identified in the requirements. For patients, it combines doctor discovery, booking, consultation, prescription access and reminders in one bilingual app with offline capability, along with AI-assisted triage and a consent-bounded conversational assistant. For doctors, it provides verified profiles, configurable schedules with optional Google Calendar sync, a structured consultation workspace, AI-generated patient summaries and analytics. For administrators, it provides doctor-verification queues, moderation tools, account management and system-health monitoring.

Beyond healthcare, the architecture and engineering patterns from this project are also transferable to other consent-sensitive domains such as legal-tech document workflows, education credentialing and digital government services, where structured workflows, document intelligence and safety-bounded AI are equally important.

1.8 Organisation of the Project

The subsequent sections of this document have been divided into six chapters. In chapter two, the relevant literature from both national and international fronts has been discussed and Medora has been positioned relative to the existing gap. The third chapter contains the discussion on methodology, which includes the architecture, data flow, role-based workflow design, and the reasons for the design of the sub-systems. Implementation details, experiment design, results, and analysis based on each of the objectives can be found in chapter four. Societal, health, environmental, safety, ethical, legal, and cultural issues have been addressed in chapter five. The sixth chapter focuses on the challenging aspects of engineering and related activities involved in the project in terms of accreditation requirements.

1.9 Reproducibility

Reproducibility plays an essential role in proving the reliability and scientific significance of any software product. The current subsection gives a detailed system architecture description, setup instructions, characteristics of datasets used in experiments, and description of the evaluation process to make reproduction of the system and experiments possible.

1.9.1 System Overview

Medora is a modular multi-service platform that comprises three main elements: frontend client app, backend API service, and AI/OCR service. Real-time database, AI-based decision support, and data pipelines based on AI are integrated in the platform. Inter-component

communication is ensured via RESTful API endpoints and synchronization methods.

1.9.2 Software Availability

The complete source code of the Medora system is maintained in a version-controlled repository. The repository includes all major modules, including frontend, backend, AI services, configuration files, and database migration scripts.

Repository: Medora **Version:** v1.0 (Final Project Release) **License:** MIT

1.9.3 System Requirements

The system requires the following software dependencies:

- Node.js (version 18 or higher)
- npm (version 9 or higher)
- Python (version 3.11 or higher)
- Supabase project (PostgreSQL, Authentication, and Storage enabled)
- Azure AI Document Intelligence credentials (for OCR processing)
- AI provider credentials (at least one of Groq, Gemini, or Cerebras)
- Docker and Docker Compose (optional for containerized deployment)

1.9.4 Installation and Setup

The system can be reproduced by following the steps below:

Repository Setup

```
git clone https://github.com/CSE-3200-System-Project/Medora.git
cd Medora
```

Frontend Setup

```
cd frontend
npm install
npm run dev
```

Backend Setup

```
cd backend
python -m venv .venv

# Activate virtual environment

# Windows:

..venv\Scripts\Activate.ps1

# Linux/macOS:

# source .venv/bin/activate

pip install -r requirements.txt
alembic upgrade head
uvicorn app.main:app --reload --port 8000
```

AI OCR Service Setup

```
cd ai_service
python -m venv .venv

# Activate virtual environment

pip install -r requirements.txt
uvicorn app.main:app --reload --port 8001
```

Environment Configuration

Environment variables must be configured for all services:

- **Frontend:** API endpoints and Supabase configuration
- **Backend:** database connection, authentication, AI provider selection, and service URLs
- **AI Service:** OCR model configuration, Azure credentials, and detection thresholds

Service Verification

System health can be verified using:

```
curl http://localhost:8000/health
curl http://localhost:8001/health
```

1.9.5 Dataset Description

Source code of the whole Medora system is stored in the version control system. The source code includes frontend, backend, AI services, settings, and database migration scripts.The evaluation OCR dataset contains anonymized and selected handwritten prescriptions collected from publicly available sources.

Due to the privacy policy, the dataset cannot be published.

Characteristics of the dataset:

- Number of samples: 50-100 prescription images
- Variation: multiple handwriting styles (clear, medium, and hard)
- Information content: medicine names, dosing and frequency

1.9.6 Experimental Reproduction

API Performance Testing

The API was tested under various load conditions with virtual users. Performance tests were run using tools like k6 to evaluate the system's latency, throughput, and error rate.

```
k6 run --vus 50 --duration 30s test.js
```

Response time, 95th percentile latency and request success rate were measured.

OCR Evaluation

OCR accuracy was measured by comparing the extracted structured data with the ground truth values. Success rate was calculated based on accurately predicting the name of the medication and the dose.

Accuracy = Correct Predictions / Total Samples

The test also uses different levels of handwriting difficulty to measure robustness.

Real-Time Synchronization Testing

Real-time processing was evaluated using propagation delay between multiple client sessions. Changes made in one client were measured in another client to approximate real-life synchronization delay.

Frontend Performance Testing

Google Lighthouse was used for assessing the frontend. The Largest Contentful Paint (LCP), Cumulative Layout Shift (CLS) and performance scores were measured in mobile simulation mode.

Qualitative Evaluation

The system was tested under typical use cases such as patient registration and appointment scheduling, AI-based doctor search, and prescription processing using OCR. The measurements are execution time, task completion rate and usability factors.

1.9.7 Reproducibility Considerations

The system is replicable using the setup described, but some system components (such as AI model responses and real-time synchronization) may vary due to external factors, network delays, and third-party services. But the approach described guarantees consistent and comparable system performance.

1.9.8 Limitations

Experiments were performed in a controlled environment with simulated traffic and a small dataset. This research did not include large-scale deployment and user variability. Potential future investigations could include a larger dataset and distributed testing.

CHAPTER 2

Related Works

2.1 Introduction

The literature for this project can be grouped into three broad categories: systems and platforms in Bangladesh, with the same language and socio-economic background as Medora; world-wide telemedicine and electronic-health-record systems from developed economies; and studies on AI-enabled clinical workflows that provide insights into safety. This chapter reviews these three, their limitations and uses these to define the research gap filled by Medora.

Further, the Bangladesh context also involves aspects such as intermittent internet connectivity, the use of two languages (Bangla-English), non-integrated care and reservations around electronic record-keeping. These are important in the evaluation of existing solutions and the development of Medora.

2.2 Related Works

2.2.1 Telemedicine in Bangladesh

DocTime is an existing telemedicine platform in Bangladesh, providing video consultations and e-prescriptions. This is a good teleconsultation app but it lacks patient medical history and AI-driven practices.

Sebaghar provides an improved consultation system with sample collection. But it lacks continuity of patient record and medication advice.

Praava Health is a hybrid online-offline service. It has its own recordkeeping systems but these are not completely mobile. Maya is great for communication in Bangla, but lacks appointment and prescription management.

2.2.2 Appointment and Doctor Listing Systems

Appointment systems such as Doctorola, GoodDoktor, Doctor Appointment BD and JOTNO are mostly focused on appointments and connecting patients with doctors. They provide detailed doctor listings, and improve access.

However, a major issue with these systems is the lack of synchronisation. Scheduling is generally asynchronous, which leads to stale availability, race conditions and conflicts. Additionally, they lack transactional integrity, and do not take patient history into account when scheduling.

Directory systems, such as Healtha, Medexly and MedicBD, are information systems that provide doctor and hospital listings, and medication databases. For instance, MedicBD is a health and directory system offering structured data.

These are good sources of information discovery but do not offer full health services, such as consultative continuity, structured data and smart support.

2.2.3 Home and Hybrid Health Services

Sasthya Seba is a successful home-based healthcare service in Bangladesh that provides services like doctors, nurses, and labs. It is centred on service orchestration, which allows patients to access a range of services.

The idea of integrated care at home inspired Medora's development. The idea of providing service discovery, scheduling and care services was drawn from platforms such as Sasthya Seba.

However, existing solutions do not have formal digital health record systems, real-time data integration and AI-powered decision support systems that limit their scalability and intelligence.

Other service discovery solutions such as Olwel and Praava Health also enable hybrid care but without workflow integration and intelligent system design.

2.2.4 Global Healthcare Platforms

Big enterprise systems such as Epic and Cerner are some of the biggest global health systems with good electronic health record systems and interoperability standards like HL7 FHIR. These are costly and not suitable for low- and middle-income countries.

Telehealth systems such as Teladoc and Zocdoc demonstrate scalable teleconsultation and scheduling systems. AI-powered systems such as Ada Health and Babylon Health show the power of intelligent diagnosis, but also the danger of hallucination and inappropriate advice.

These examples point to the need for safe-bounded AI, which Medora offers.

Table 2.1: *How Medora compares to existing healthcare platforms across the features that matter.*

Platform	Structured	Real-time	OCR	AI	Consent	Workflow	Bangla
DocTime	Partial	No	No	No	No	Partial	Partial
Sebaghar	No	Partial	No	No	No	Partial	Yes
Praava Health	Yes	No	No	No	Partial	Partial	Yes
Doctorola	No	No	No	No	No	No	Partial
GoodDoktor	No	No	No	No	No	No	Partial
MedicBD	No	No	No	No	No	No	Yes
Sasthya Seba	No	Partial	No	No	No	Partial	Yes
Epic/Cerner	Yes	Partial	Partial	No	Partial	Yes	No
Teladoc	Partial	Yes	No	Partial	Partial	Partial	No
Ada Health	Partial	No	No	Yes	No	No	No
Medora	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

2.2.5 Document-Intelligence Systems for Prescriptions

OCR of prescriptions remains challenging, especially for handwritten prescriptions such as those used in Bangladesh. OCR services such as Microsoft's Azure Document Intelligence, Google Document AI and PaddleOCR are good places to start.However, problems with diverse handwriting, multilingual characters and poor layout affect their performance. Medora uses object detection, multi-engine OCR fallback and fuzzy matching to a list of medicines.

2.2.6 Conversational Clinical Assistants

Conversational AI has been applied with success in health care but can be dangerous if unchecked. Problems such as hallucination and overconfidence have been reported.Recent techniques address these issues through retrieval-augmented generation, schema matching, and confidence filtering. Medora adopts these techniques to enable safe and trustworthy AI-supported interactions.

2.3 Discussion

Our review reveals existing systems in Bangladesh and globally cater to particular health care needs. Telemedicine systems enable consultations but no records. Appointment systems book appointments but lack real-time consistency and integration. Directory systems provide information but lack workflows.Integrated service systems such as Sasthya Seba demonstrate the benefits of integrated service delivery and informed Medora's approach. However, there is no system offering integrated medical history, real-time scheduling, prescription intelligence, safe AI and consent-driven access to data.

CHAPTER 3

Methodology

3.1 Introduction

The Medora design process is a combination of service decomposition, contract-first API design, asynchronous backend processing and empirical validation. The system is deliberately split into a Next.js frontend, FastAPI core backend and FastAPI AI OCR microservice because the workloads, scaling characteristics and risks of the sub-systems are different. The frontend needs to balance responsiveness, accessibility and progressive-web-application features; the backend needs to orchestrate asynchronous transactional health-care workflows with role-based access controls; and the OCR microservice needs to handle the processing of documents that is intensive on CPU resources without impacting user-facing APIs. The platform services are Supabase PostgreSQL, authentication, storage and Realtime channels and the backend AI orchestrates multiple large-language-model providers through schema-validated adapters.

This chapter is based on the project documentation in the `docs` folder. Architecture, workflows, routes, database, features and tests were cross-referenced to ensure that the report displays the design of the current system, rather than just the idealistic design. Thus, the methodology below not only describes what the system should do, but how the current implementation is structured, isolated and tested.

3.2 Detailed Methodology

3.2.1 System Architecture

Medora follows a service-oriented architecture with explicit interface boundaries between presentation, transactional business logic and AI-assisted extraction. The frontend communicates with the backend over authenticated HTTPS. The backend maintains the authoritative workflow state for booking, consultation, consent, reminders, notifications and AI features. The OCR microservice is called over an internal HTTP channel only when prescription or medical-report extraction is required. Supabase provides PostgreSQL persistence, storage buckets, authentication and Realtime slot propagation, while external AI providers are accessed only through the backend orchestration layer.

This separation was chosen for three practical reasons. First, OCR workloads are com-

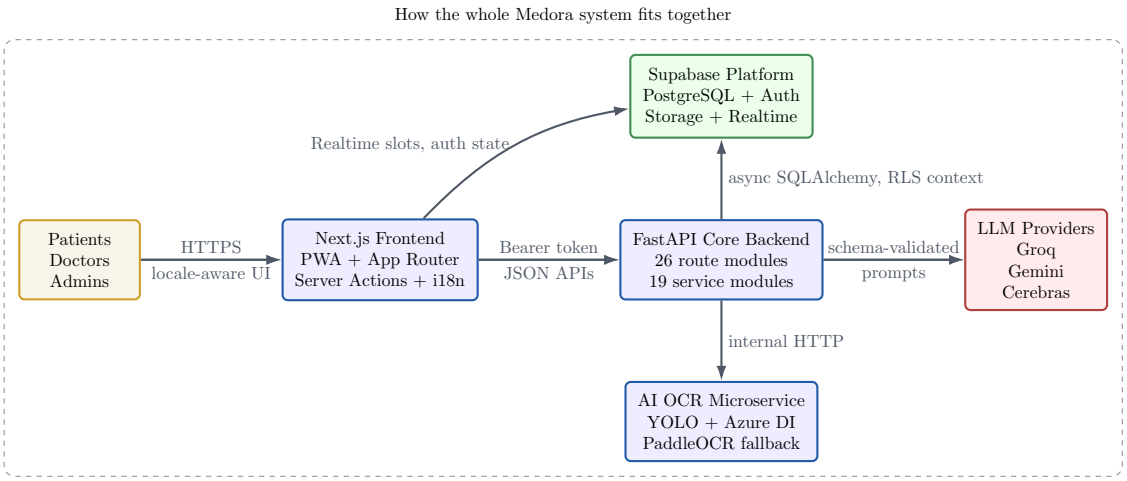


Figure 3.1: *Top-level Medora system architecture showing the frontend, backend, OCR service, Supabase platform services and controlled AI-provider boundary.*

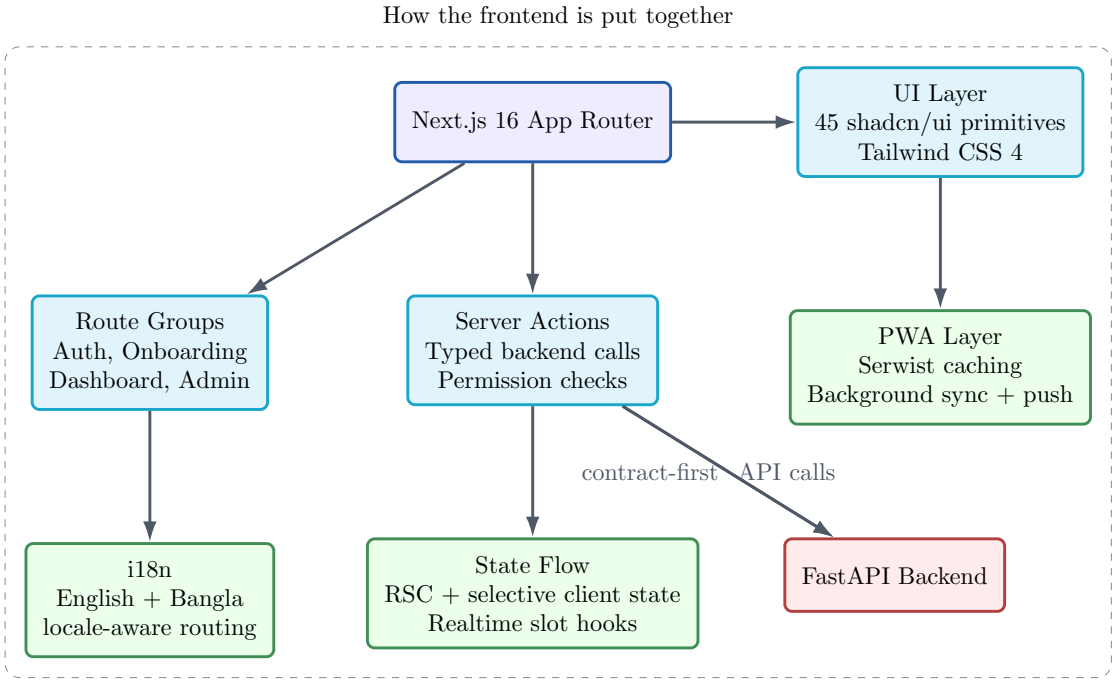


Figure 3.2: *Frontend architecture with route groups, server actions, UI primitives, localization and PWA-specific behavior.*

putationally variable and can introduce latency spikes if they share the same runtime as booking and consultation APIs. Second, the OCR service requires heavier model artifacts and image-processing dependencies than the core backend. Third, isolating the AI and OCR paths simplifies fault containment: if OCR is degraded, the booking, onboarding and notification features remain available.

The same backend AI boundary also supports natural-language doctor discovery. Let q denote a patient complaint written in everyday language and let $\mathscr{K}$ denote the set of supported medical specialities. Consistent with the documented specialty-matching flow, the first-stage

resolver can be written as

$$\hat{k} = \arg\max_{k \in \mathcal{K}} \phi(q,k),$$

where $\phi(q,k)$ is the semantic compatibility between the patient query and speciality k. After that stage, only verified doctors with speciality $\hat{k}$ are retained. The documented ranking factors—years of experience, consultation fee and location proximity—can then be expressed

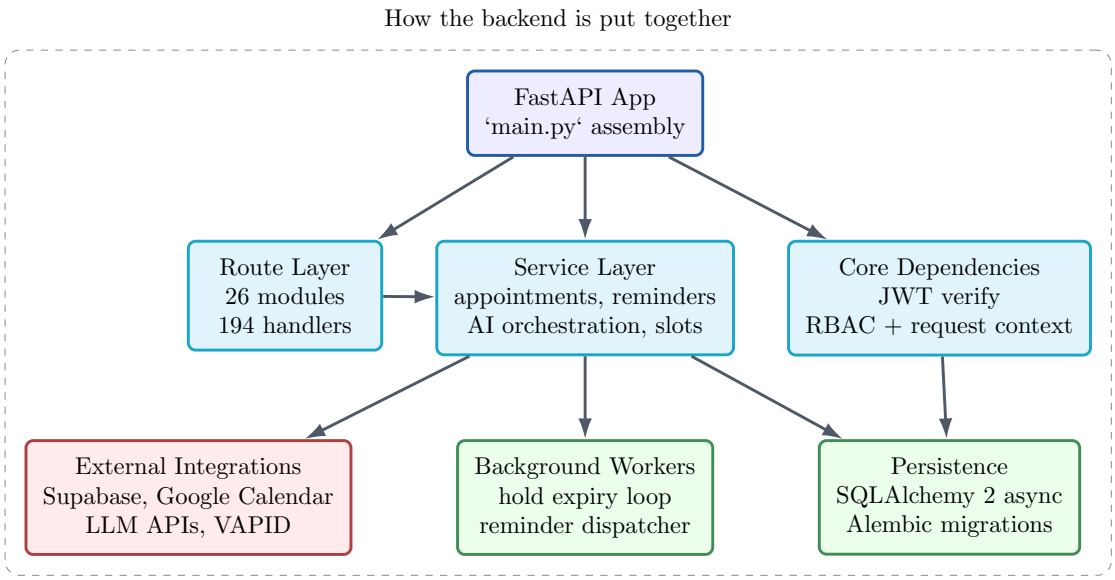


Figure 3.3: *Backend architecture showing route modules, service layer, security dependencies, persistence and background workers.*

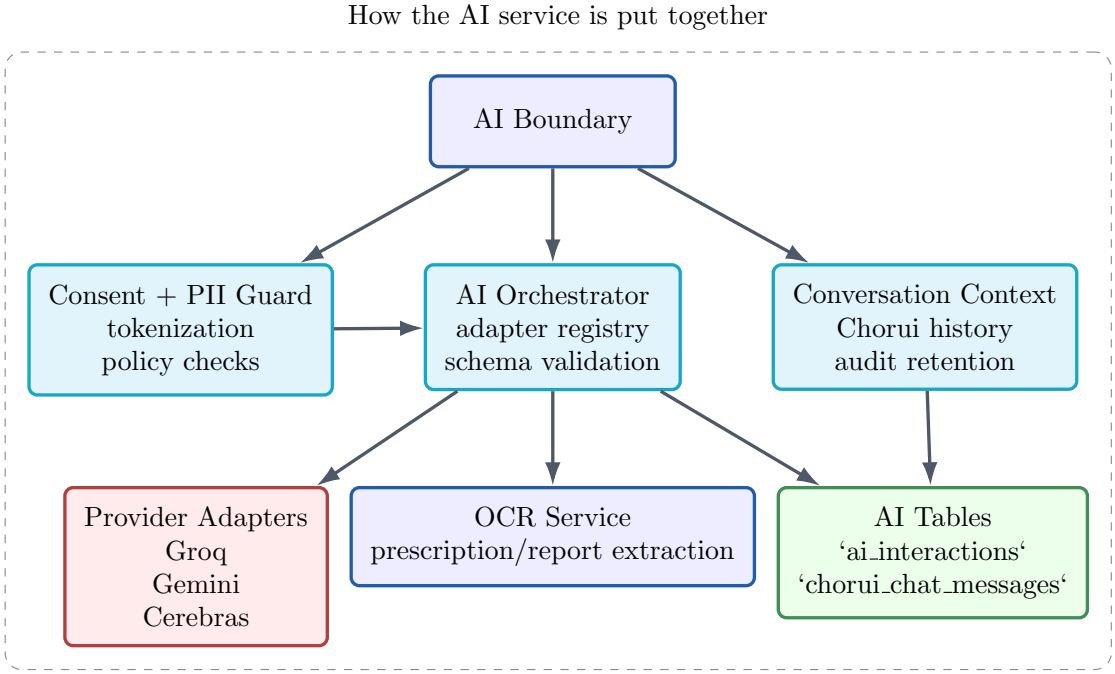


Figure 3.4: *AI-service architecture with consent checks, provider adapters, schema validation, OCR linkage and conversation history.*

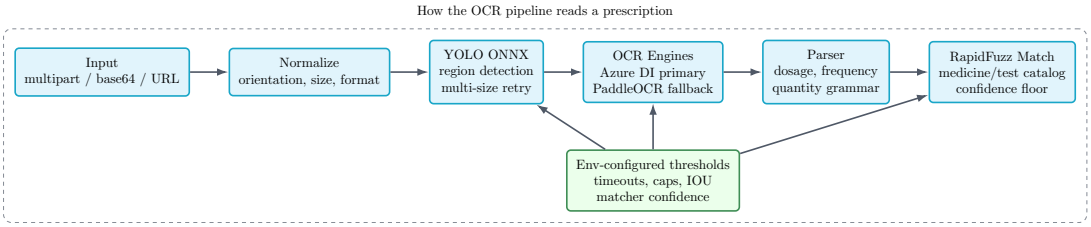


Figure 3.5: *OCR pipeline from upload and preprocessing through detection, OCR, parsing and structured extraction.*

transparently as

$$R(d \mid q,\ell) = w_e \widetilde{E}(d) + w_f\big(1-\widetilde{F}(d)\big) + w_\ell\big(1-\widetilde{\Delta}(d,\ell)\big), \qquad w_e + w_f + w_\ell = 1,$$

where $\widetilde{E}(d)$, $\widetilde{F}(d)$ and $\widetilde{\Delta}(d,\ell)$ are min–max normalized experience, fee and patient-to-doctor distance terms, and ℓ is the patient location context when available. Writing the ranking in this way helps readers see that AI doctor search is not a black-box recommendation step; it is a speciality-resolution stage followed by an interpretable multi-factor ordering over verified doctors.

If the ranked list needs an explicit confidence measure for presentation, the same speciality-matching stage can be turned into a normalized distribution,

$$P(k \mid q) = \frac{\exp\big(\beta\, \phi(q,k)\big)}{\sum_{k' \in \mathscr{K}} \exp\big(\beta\, \phi(q,k')\big)}, \qquad c_{\text{search}}(q) = \max_{k \in \mathscr{K}} P(k \mid q),$$

where $\beta > 0$ controls how sharply the top speciality separates from competing alternatives. This makes it possible to distinguish between confident speciality resolution and cases that should fall back to broader manual search.

3.2.2 OCR Pipeline Architecture

The OCR microservice is a separate FastAPI application with dedicated endpoints for prescription extraction and medical-report extraction. Inputs may arrive as multipart form uploads, base64 payloads or external URLs. The pipeline first normalizes image orientation, size and format. Region proposals are then produced with a YOLO ONNX detector. When confidence is insufficient, the system retries at alternate scales. Cropped regions are submitted to Azure Document Intelligence as the primary OCR engine, while PaddleOCR acts as a fallback if Azure is unavailable or constrained.

The DFD hierarchy makes the concurrency-sensitive parts of the platform visible. Booking depends on soft holds, transactional commit and state broadcast. OCR depends on a staged extraction path with fallback. The backend acts as the control plane that decides which stores

are read or written and when user-visible notifications should be emitted.

3.2.3 Workflow Designs

The principal user journeys were designed as deterministic workflows rather than loosely coupled page transitions. This is most important in booking, consultation and AI-assisted navigation, where multiple services must coordinate without ambiguity.

Appointment booking is a coordination workflow. The patient looks for a doctor, the client subscribes to real-time updates on available slots, the server applies constraints and places a soft hold, and the transaction is committed to reserve the appointment and send notifications.

The Chorui workflow takes a deterministic approach to conversation. A voice request is interpreted, contextualized, validated and then directed to an approved path or response. Confidence levels determine whether the system can proceed, must seek confirmation or needs clarification. The Vapi voice workflow introduces a voice session manager and a cloud-based voice agent, but it also follows the same principles of control: all health-related reasoning is deferred to Medora's own systems, and any local client action is followed by a validated action in the back end.

3.2.4 Use-Case and Role Feature Views

The provided inventory of features implemented is so large that role-based perspectives should be used to make it clear. Onboarding, doctor discovery, bookings, reminding, OCR uploads and Chorui are the main points of interaction between patients. Physicians are concerned with scheduling, consultations, prescriptions, reviewing patient data and support in the calendar-based workflow. Administrators are in charge of verification, moderation and monitoring and

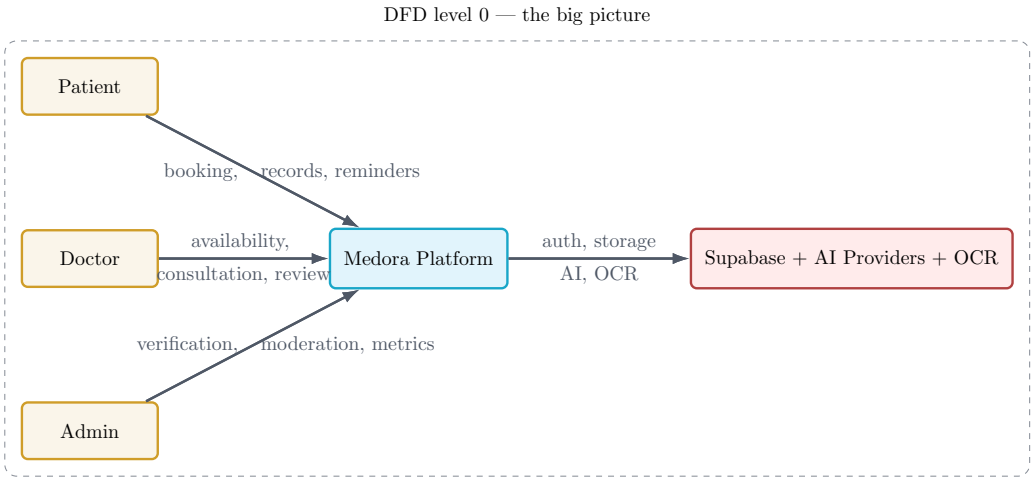


Figure 3.6: *Context-level data-flow view showing how patients, doctors and administrators interact with the platform and its dependent services.*

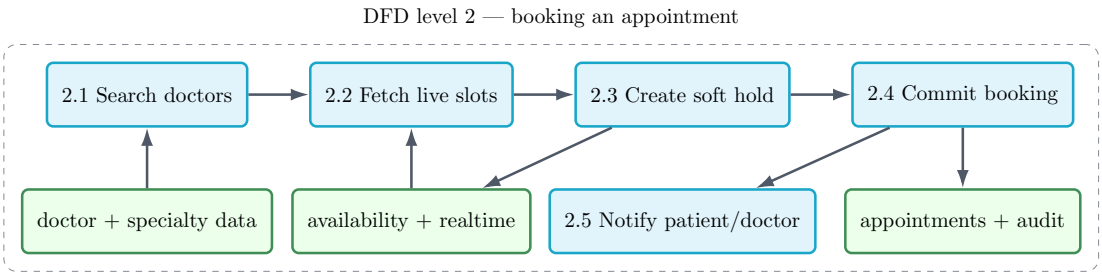


Figure 3.7: *Detailed booking data flow including search, live slot retrieval, hold placement, final commit and notification generation.*

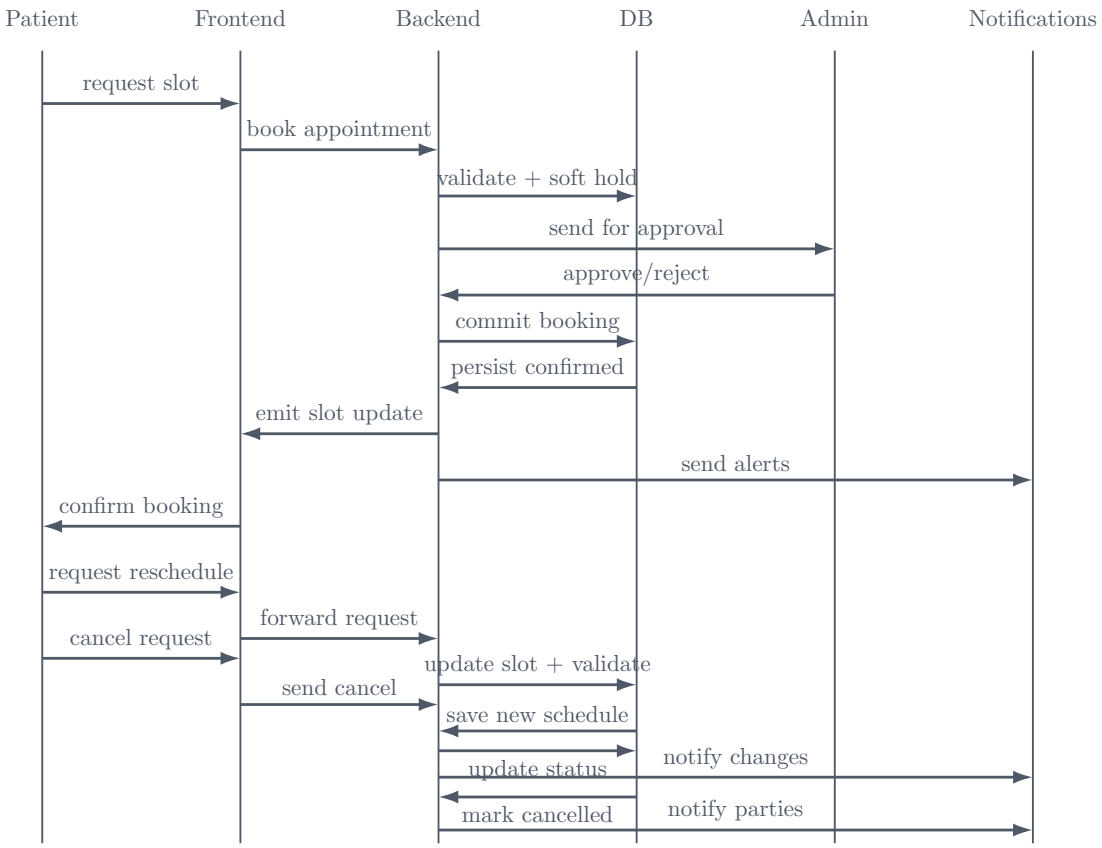


Figure 3.8: *Sequence view of appointment booking, including live slot synchronization, hold placement, booking confirmation and notifications.*

governance activities. The cross-reading of the frontend route inventory and the backend route catalog, as well as the appointments documentation and the consolidated features inventory in the `docs` folder, included the use-case views below, such that the diagrams represent the product surface implementation as opposed to an imagined requirements list.

In the case of the patient, the key use cases begin with progressive onboarding and then progress to doctor search, booking and longitudinal care management. The feature list and the route map reveal that this role is not limited to creating appointments: the patient has access to search (with specialty and district filters), inspect the profile, use map-based location context, upload reports (to extract OCR-data), manage reminders and notifications, see the

list of previously visited doctors (after making appointments), and use Chorui as a guidance system. The appointments documentation further demonstrates that the booking is linked to the live slot state, cancellation and rescheduling workflows, generation of reminders and post-visit feedback, and thus the patient use-case view focuses on a journey instead of a one-off transaction.

For patients, the feature map condenses several subsystems that are otherwise scattered across the architecture. Signup and onboarding establish role, locale and baseline medical context. Doctor discovery combines structured filtering with AI-assisted specialty matching and review-informed reputation cues. Appointment booking depends on realtime slot synchronization and hold-based confirmation. Prescription and report OCR connect document upload to extraction and structured review. Reminders, medical-history follow-up and previously-visited-doctor review entry points extend the lifecycle past booking, while Chorui offers a bounded navigation and assistance layer over the same feature set. This summarization helps explain why the patient interface in the implementation chapter spans both clinical and non-clinical views.

For doctors and administrators, the feature map shows a clean separation between operational medicine and platform governance. The doctor workspace combines schedule management, consultation editing, patient-report review, calendar synchronization, reminders and AI assistance into one verified-professional surface. The admin console instead concentrates verification, moderation, metrics and audit functions. This distinction is methodologically important because it explains several later implementation choices: separate route groups, different authorization dependencies, different dashboard widgets and different evidence trails in the database.

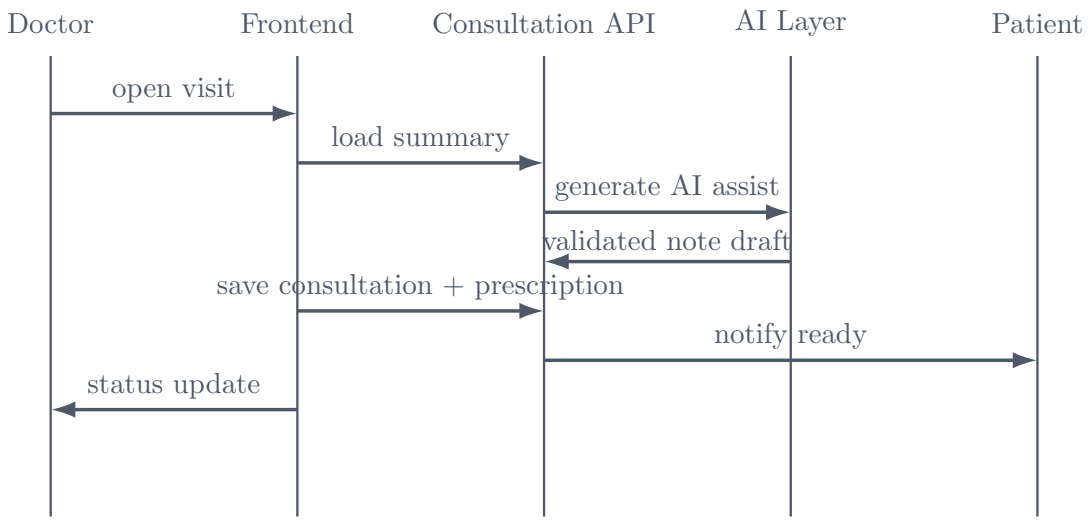


Figure 3.9: *Consultation sequence showing patient-context retrieval, AI-assisted drafting, persistence and downstream patient-facing outputs.*

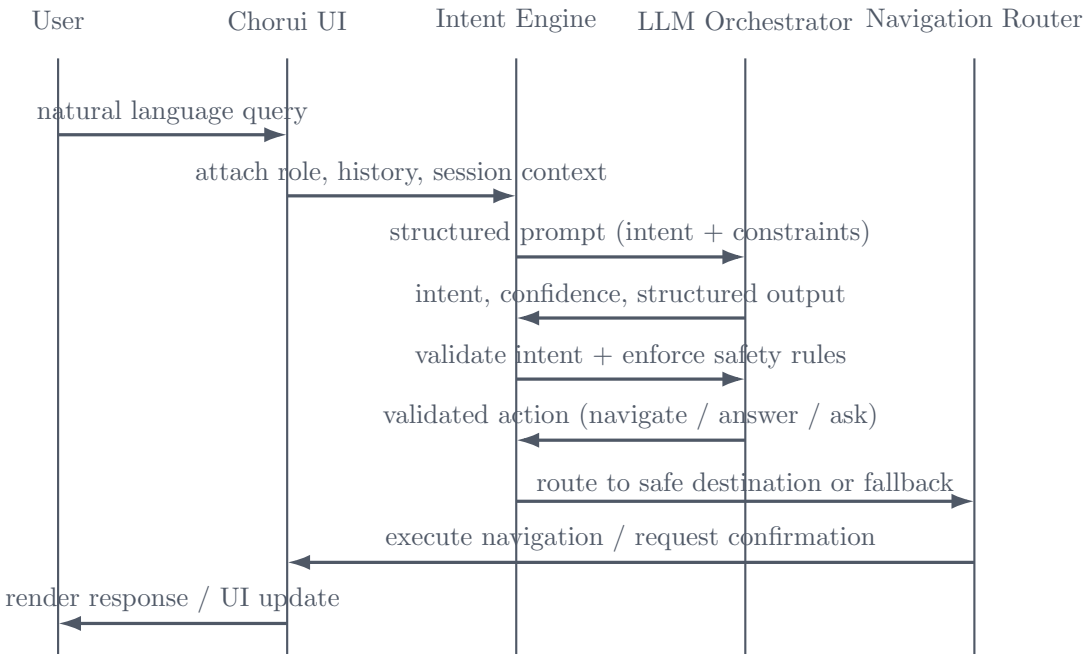


Figure 3.10: *Safe conversational flow for Chorui, from user intent through orchestration and bounded response generation.*

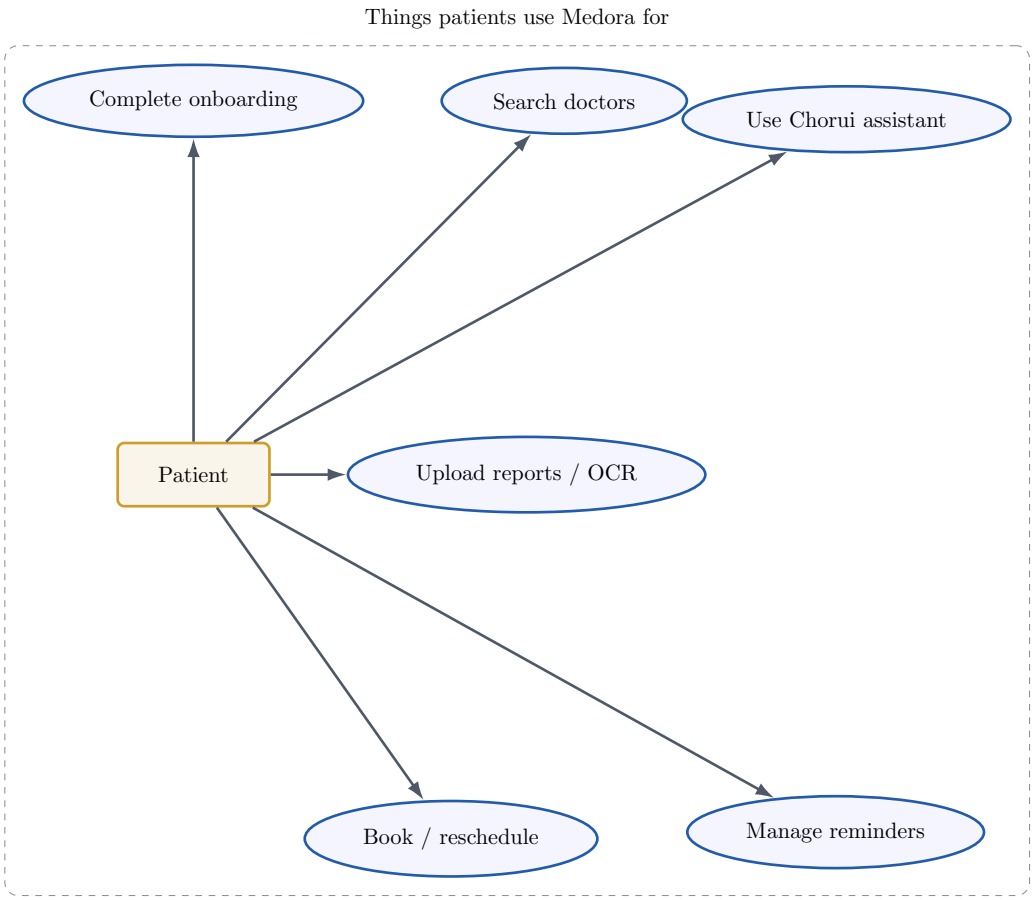


Figure 3.11: *Patient use-case diagram summarizing the primary journey from onboarding and search to booking, records, reminders and assistance.*

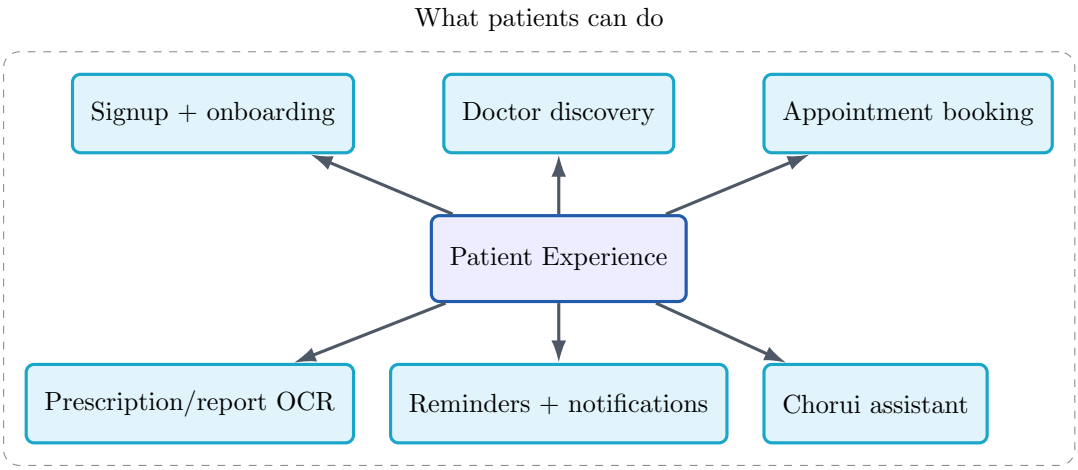


Figure 3.12: *Patient feature map showing how discovery, booking, OCR, reminders and follow-up functions are grouped in the product surface.*

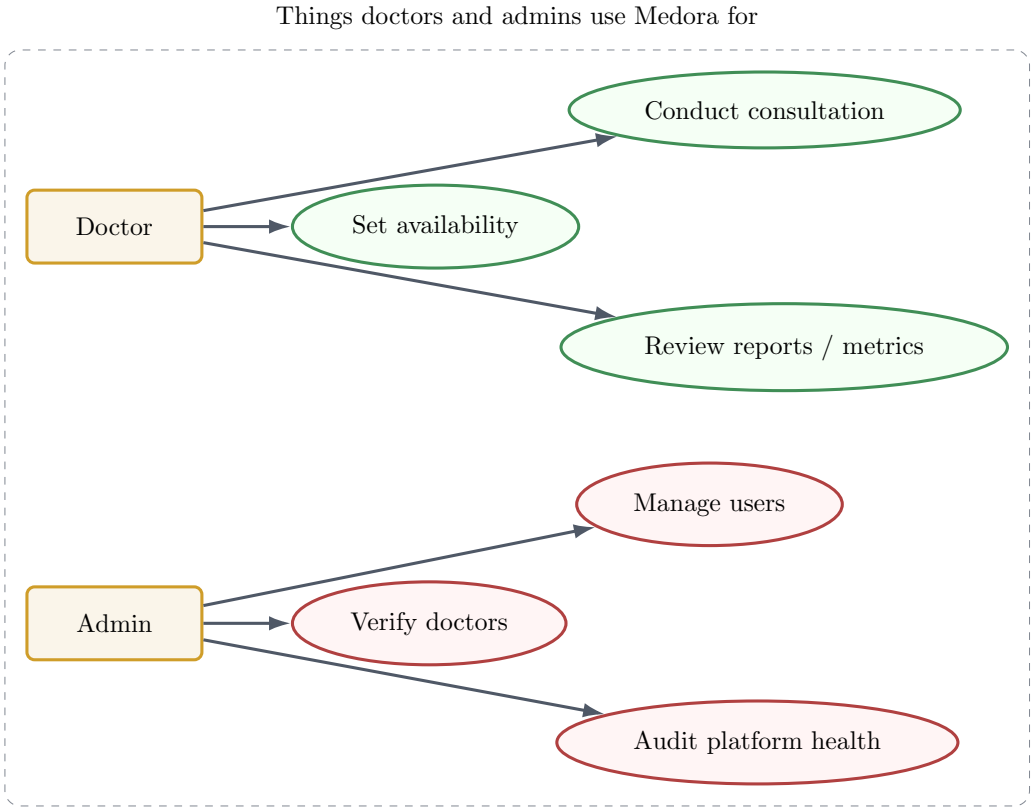


Figure 3.13: *Combined use-case diagram for the doctor and administrator roles, highlighting the split between clinical workflows and governance workflows.*

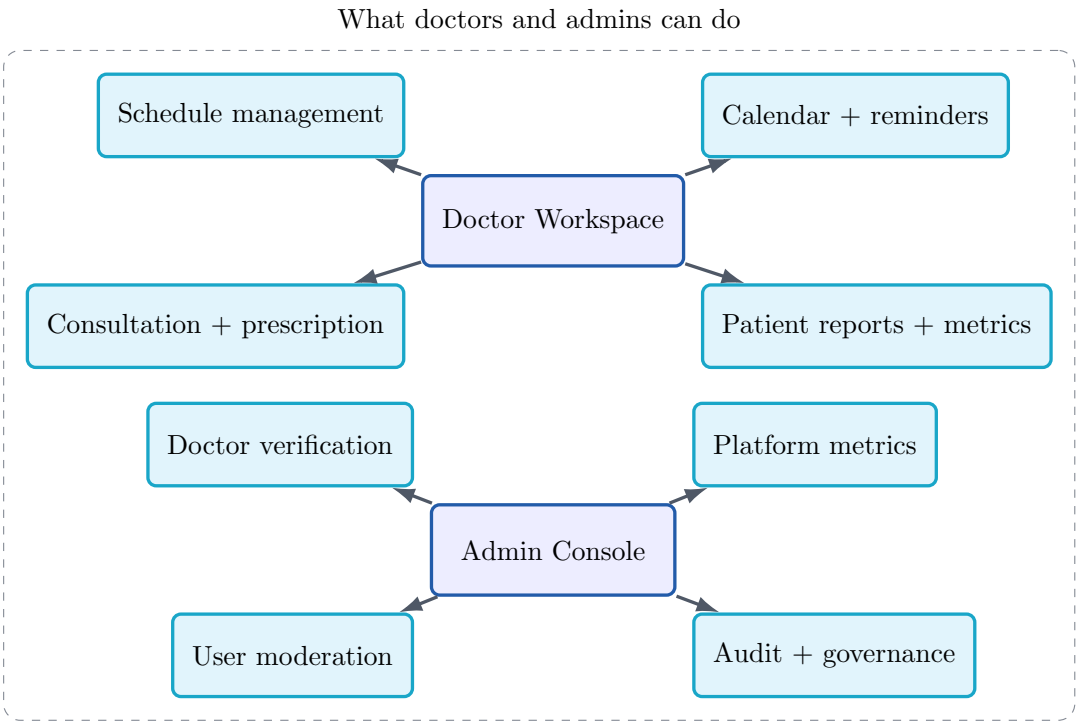


Figure 3.14: *Feature map for doctor and admin surfaces, showing how consultation, scheduling, verification and moderation are partitioned.*

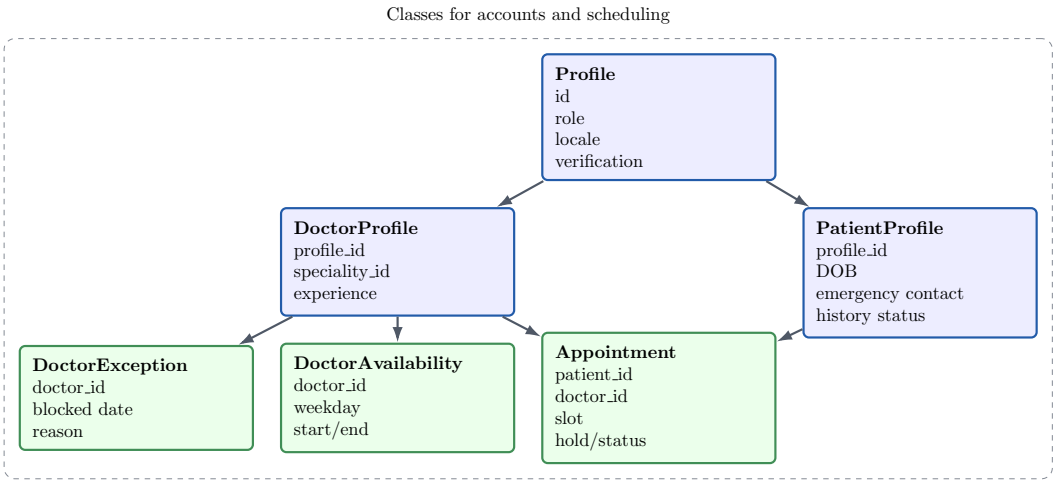


Figure 3.15: *Class-level view of identity, scheduling and appointment structures used by the core transaction flows.*

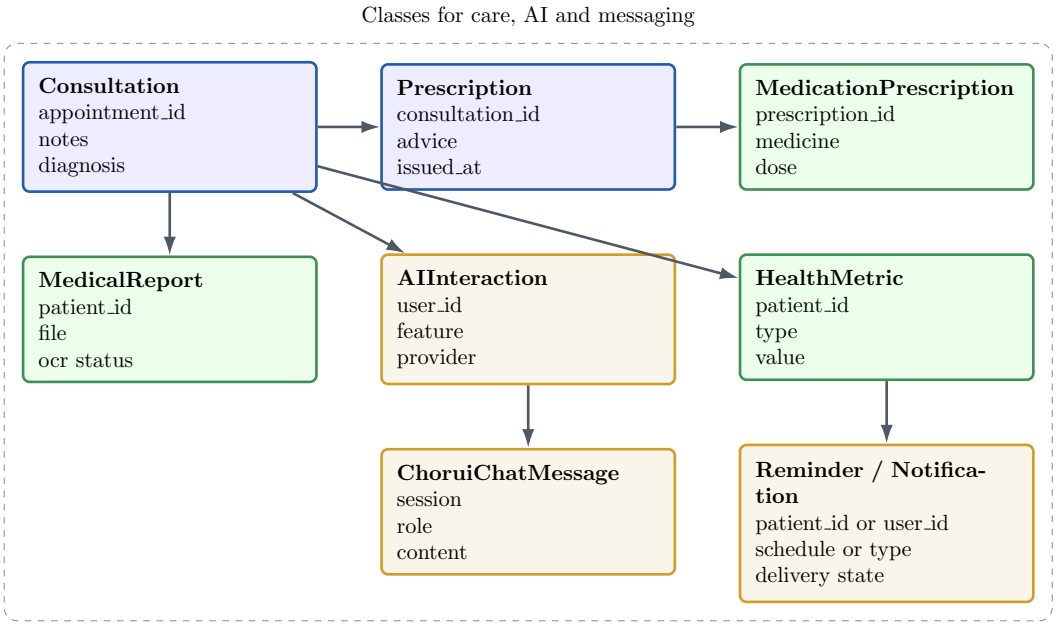


Figure 3.16: *Class-level view of clinical records, OCR artifacts, AI interactions and related consultation structures.*

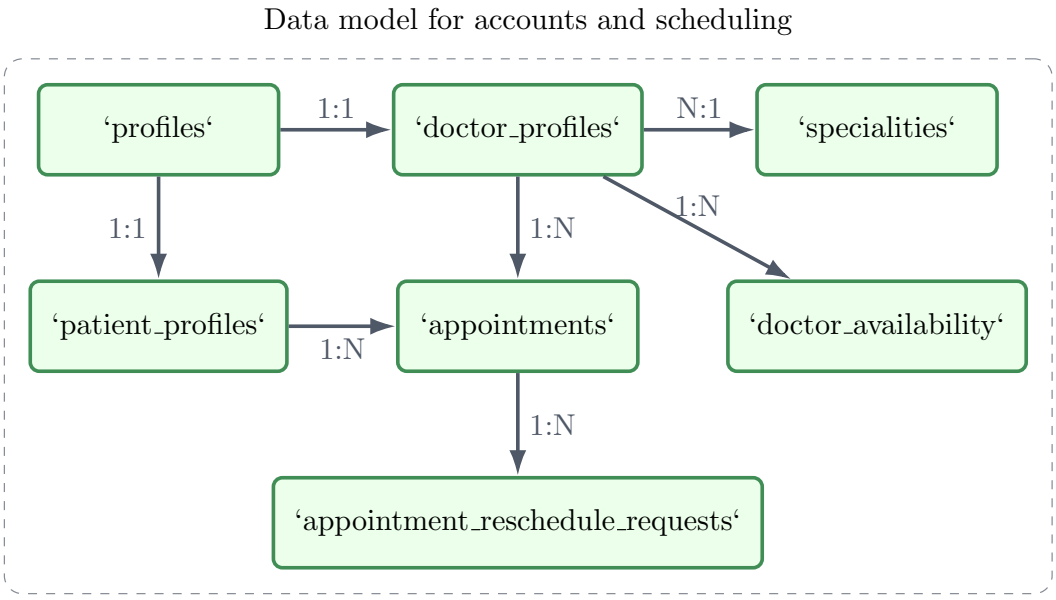


Figure 3.17: *Entity-relationship view of identity, profiles, availability and appointment scheduling data.*

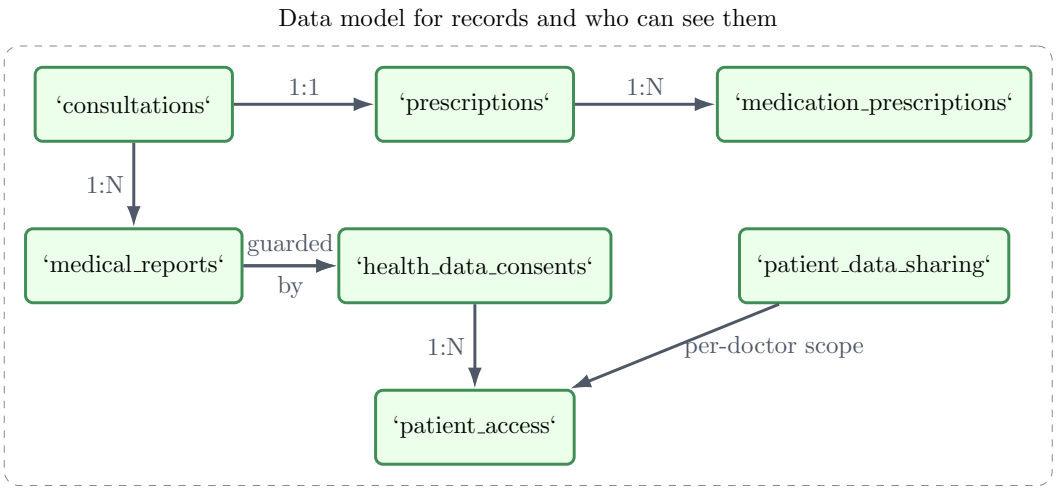


Figure 3.18: *Entity-relationship view of clinical data, record access and consent-aware sharing structures.*

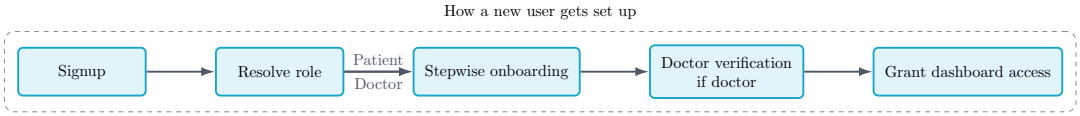


Figure 3.19: *Activity view of onboarding, including the additional verification gate used for doctor accounts.*

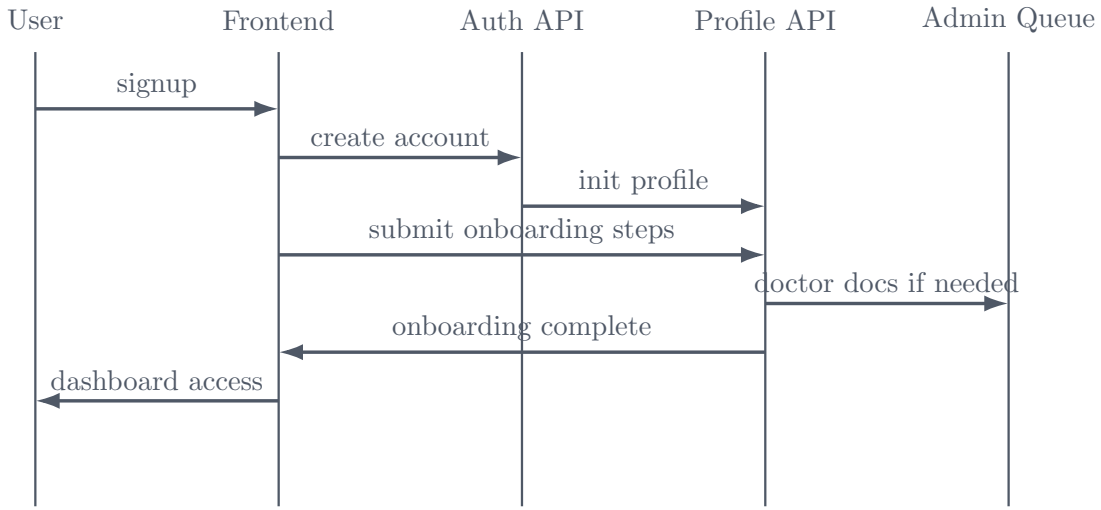


Figure 3.20: *Onboarding sequence spanning user registration, profile creation, verification and role-dependent access.*

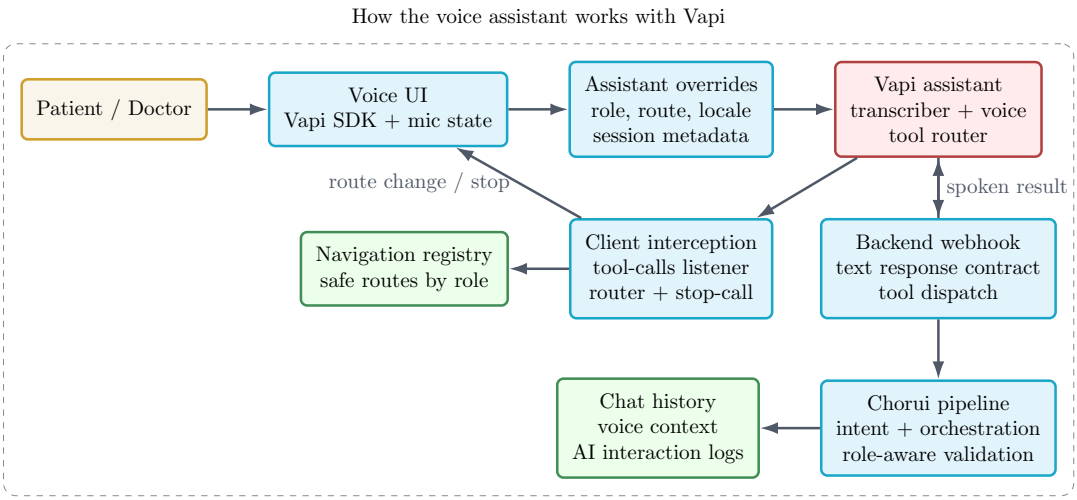


Figure 3.21: *Voice-assistant architecture showing session management, tool routing and the boundary between local and backend actions.*

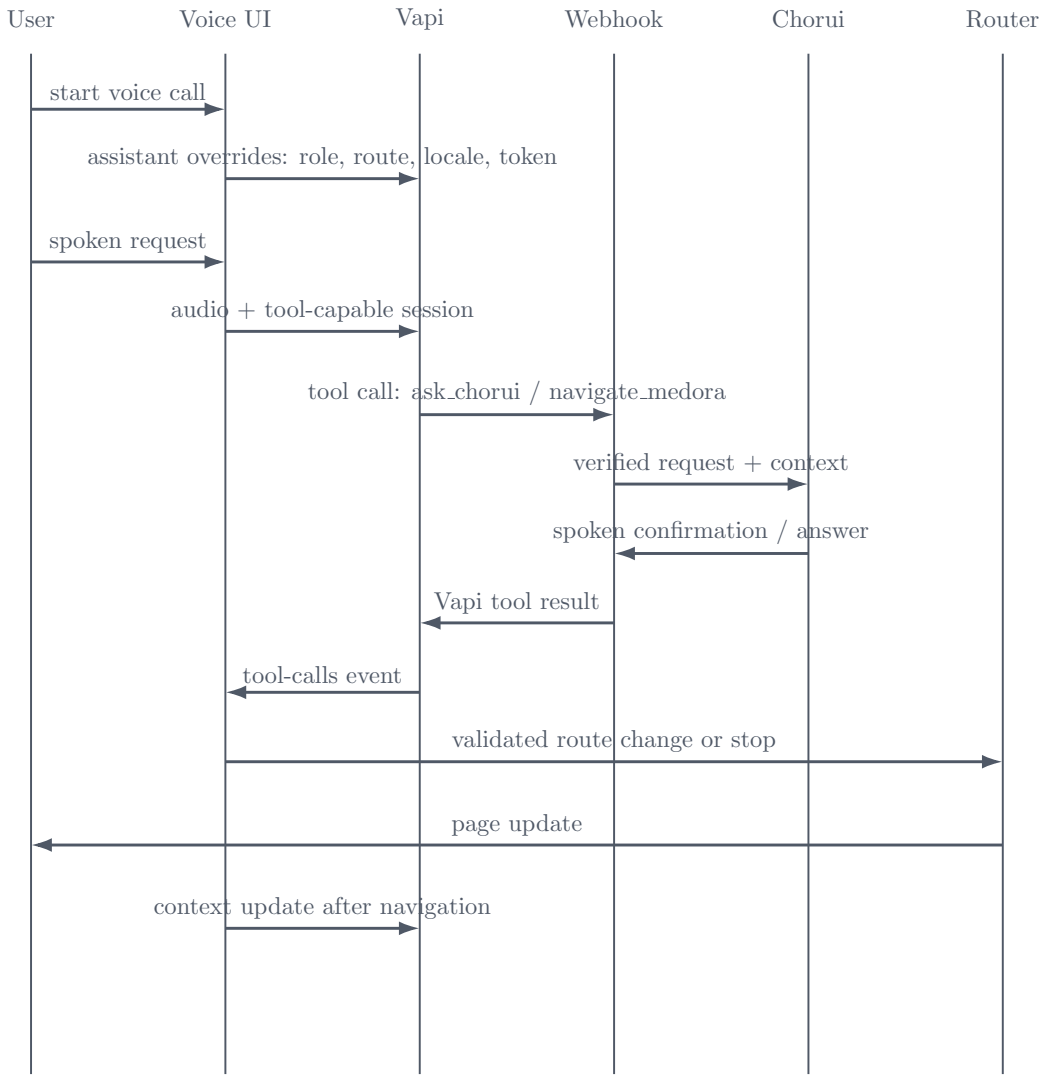


Figure 3.22: *Voice-assistant sequence showing tool invocation, backend execution and refreshed context after action completion.*

3.3 Conclusion

The methodology makes Medora a production-based, modular healthcare platform and not a monolithic student prototype. Isolation of services shields main workflows against AI variability and OCR load. The frontend is designed to support route safety, server behaviors, accessibility and progressive-web-application behaviors. The backend focuses on the workflow management, security and on-demand business logic. Consent, privacy and schema validation identify the AI layer. The OCR service is configured, monitored and observable. Different focused schema, data-flow, sequence, activity and use-case views were employed as opposed to large dense diagrams so that the system can be readable and reviewable. The following chapter introduces the implementation evidence that proves the validity of these architectural decisions, including the testing strategy, and quantitative results.

CHAPTER 4

Implementation, Results and Discussions

4.1 Introduction

This chapter translates the methodology into implementation evidence and measured outcomes. The emphasis is on showing how the architecture described in Chapter III appears in concrete modules, workflows, tests and performance results. The same project documentation used for the methodology chapter was also used here to align implementation counts, route groups, test scopes and benchmark targets with the actual system organization.

4.2 Experimental Setup

Development and evaluation were performed in a mixed local-and-cloud environment. Local development machines typically provided eight CPU cores, 16 GB RAM and SSD storage, while staging deployments ran on containerized infrastructure with independent scaling for the frontend, backend and OCR service. The backend was configured to scale on CPU and concurrent request pressure, and the OCR service scaled more aggressively on CPU-bound workloads. PostgreSQL persistence, authentication, object storage and Realtime channels were provided by Supabase.

The evaluation tooling mirrored the system boundaries. API and workflow load were exercised with Locust and k6. Frontend quality was tracked with Lighthouse CI and bundle-size checks. Backend and integration behavior were verified with automated unit, integration, end-to-end and security tests. OCR evaluation used a labeled prescription set and a curated medicine catalog for entity matching. AI-provider evaluation focused on structured-output compliance rather than unrestricted text quality because the platform depends on schema-valid responses.

4.3 Evaluation Metrics

The main quantitative metrics were selected from the platform's operational risks and non-functional goals. Backend metrics included API latency at the 50th, 95th and 99th percentiles, reminder and slot-update responsiveness, and database query latency under concurrency. OCR metrics included extraction latency and structured-field accuracy. Frontend metrics

included Largest Contentful Paint and Cumulative Layout Shift. Validation metrics included end-to-end journey completion, security control enforcement, and regression resistance in CI. These metrics were chosen because they directly reflect the perceived quality of booking, consultation, report upload and guided-assistant flows.

4.4 Dataset

Three datasets or workload sources were used repeatedly during evaluation. The first was a synthetic scheduling workload containing many patient and doctor actors over a multi-week horizon so that booking, reschedule and cancellation paths could be stressed concurrently. The second was an OCR corpus of 100 prescription images with field-level annotations for medicine names, dosage and frequency. The third was a medicine catalog of roughly 20,000 entries used to support OCR normalization and fuzzy matching. Together these datasets allowed the team to evaluate transactional correctness, OCR usefulness and search-quality support within the same platform.

4.5 Implementation and Results

4.5.1 Subsystem Implementation Evidence

The frontend application is built according to the published Next.js App Router framework, and has separate route groups in authentication, onboarding, dashboards and administration. The access layer of the data is constructed over server operations instead of arbitrary callouts by the client to fetch data, which ensures that permission checks are near the backend contract. Reusable UI primitives are used to ensure consistency between patient and doctor and admin views, and Serwist is used to enable caching, background synchronisation and push behaviour required in using progressive-web-applications.

The backend implementation focuses on FastAPI application assembly in `app/main.py`, security and dependency modules, route modules and service modules. The documentation lists 26 route files and 19 service modules, which is reflective of a backend that focuses on domain orchestration, not route-local business logic. Startup hooks are used to preload chosen AI resources, fix known instances of schema drift and initiate the background loops that expire booking holds and send reminders.

The desired separation of concerns can also be seen in the AI implementation. Backed orchestration layer implements provider abstraction, role-sensitive context preparation, consent checks, PII tokenization and schema validation. The OCR service is a separate boundary of extraction that can adjust its model thresholds, OCR providers and fallback behavior without disrupting the booking and consultation runtime. Chorui is an interface that is mounted on

top of these layers as a limited conversational interface over a free-form chatbot.

However, one specific design choice merits further elaboration, as it applies to every external AI API invocation: Medora never transmits unprotected raw person-identifying context to an external model provider. If the structured context is denoted by $x = \{(k_i, v_i)\}_{i=1}^{n}$, the corresponding active sharing preferences are indicated by Π_u, and the function $\mathrm{cat}(k_i)$ retrieves the category from the context dictionary item k_i, then consent pruning may be expressed as

$$F(x, \Pi_u) = \{(k_i, v_i) \in x \ : \ \mathrm{cat}(k_i) \in \Pi_u\}.$$

Once all categories lacking explicit patient consent have been stripped, anonymisation follows by means of the following sanitization operator $S(\cdot)$:

$$S(k_i, v_i) = \begin{cases} \big(k_i, [\text{redacted-}\tau(k_i)]\big), & k_i \in K_{\mathrm{PII}}, \\ \big(k_i, \rho(v_i)\big), & k_i \notin K_{\mathrm{PII}}, \end{cases}$$

where K_{PII} is the list of forbidden field names like name, email, phone, national identifier, address and date of birth, $\tau(k_i)$ provides the corresponding anonymization label, and $\rho(\cdot)$ applies regular expressions to eliminate all remaining patterns of identifiers.

For each identifiable piece of information z in a certain namespace ν (such as the patient, doctor or report subject), a stable pseudonymous identifier token is emitted instead of the original value:

$$T_\nu(z) = \big[\text{anon} : \nu : \mathrm{trunc}_\ell(H_{K_\nu}(z))\big],$$

where $H_{K_\nu}(\cdot)$ denotes a keyed cryptographic hash function that uses a secret known to Medora only, and $\mathrm{trunc}_\ell(\cdot)$ reduces the token size to keep the overhead low. As a consequence, the payload transmitted to the external provider takes the following form:

$$q_{\mathrm{out}} = P\Big(S\big(F(x, \Pi_u)\big)\Big),$$

where $P(\cdot)$ is the prompt creation function. What Medora's external partners get, in other words, is filtered-by-consent, devoid-of-PII and hash-identified content without access to the underlying real-world identities. The same idea holds for OCR-side context tracing via the anonymised subject token header rather than user identifier or even a signed JSON Web Token (JWT).

The process flow for this process is thus,

$$x \xrightarrow{F_{\Pi_u}} x^{(c)} \xrightarrow{S} x^{(s)} \xrightarrow{P} q_{\mathrm{out}} \xrightarrow{\text{provider API}} r \xrightarrow{V} \hat{r},$$

where $V(\cdot)$ stands for validating the provider response according to the appropriate schema

before accepting the results and storing them in either application logic or AI interaction log. As far as the academic relevance of this flow is concerned, anonymity is a necessary transformation process flow and not a mere wrapping process.

To keep the equations readable in this section, Appendix A gathers the symbol definitions, normalization conventions and scope notes used by the OCR, doctor-search and analytics formalizations below.

The prescription-OCR path required an additional layer of deterministic post-processing because raw document text alone was not sufficiently reliable for medicine and dosage structuring. Let x_f denote the raw OCR dosage-frequency token. The matcher constrains the search space to the clinically meaningful three-slot daily patterns

$$\mathscr{P} = \{1{+}0{+}0,\ 0{+}1{+}0,\ 0{+}0{+}1,\ 1{+}1{+}0,\ 1{+}0{+}1,\ 0{+}1{+}1,\ 1{+}1{+}1\},$$

where each position represents morning, afternoon and evening intake respectively. A normalization operator $N(\cdot)$ first rewrites common OCR confusions such as separator substitutions ($-, |, ,$ interpreted as $+$) and glyph confusions ($l, I, | \mapsto 1$, $O \mapsto 0$). The corrected dosage pattern is then chosen as the nearest valid element,

$$\hat{p} = \arg\min_{p \in \mathscr{P}} d_{\mathrm{L}}\big(N(x_f), p\big),$$

where d_{L} is the Levenshtein edit distance over the normalized token. Once $\hat{p}$ is accepted, the implied daily frequency is computed as

$$f(\hat{p}) = \sum_{i=1}^{3} \hat{p}_i,$$

which directly yields the number of administrations per day.

Quantity extraction is formalized similarly. For a raw quantity expression x_q, numeral normalization maps Bangla digits and written English/Bangla number words to an integer n, while unit parsing identifies $u \in \{\text{day}, \text{week}, \text{month}\}$. The standardized duration in days is then

$$D(x_q) = n\,\kappa(u), \qquad \kappa(\text{day}) = 1,\ \kappa(\text{week}) = 7,\ \kappa(\text{month}) = 30,$$

with days used as the default unit when a standalone number is detected. This rule lets expressions such as “10 days”, “2 weeks”, “ten days” and the Bangla phrase “ek mash” be reduced to one comparable temporal representation.

Medicine-name recovery uses thresholded fuzzy retrieval over the catalog. Let x_m be the OCR medicine token and let $\psi(\cdot)$ denote lowercase and character-cleaning normalization

applied to both generic and brand names. The best candidate is selected by

$$m^* = \arg\max_{m \in \mathcal{M}} s(\psi(x_m), \psi(m)),$$

where $s(\cdot,\cdot) \in [0,1]$ is the RapidFuzz similarity score over the normalized strings. A match is accepted only when

$$s(\psi(x_m), \psi(m^*)) \geq \tau_m,$$

with the implementation default $\tau_m = 0.7$. This combination of constrained dosage grammar, unit-normalized duration parsing and thresholded fuzzy medicine retrieval explains why the OCR pipeline can recover structured prescriptions even when the source image contains mixed Bangla–English notation and moderate recognition noise.

The matcher described in the project documentation can be made even more explicit by writing its name-similarity stage as the piecewise score

$$s_{\text{name}}(x_m, m) = \begin{cases} 1.0, & \psi(x_m) = \psi(m), \\ 0.9, & \psi(x_m) \subseteq \psi(m) \text{ or } \psi(m) \subseteq \psi(x_m), \\ \text{RF}(\psi(x_m), \psi(m)), & \text{otherwise}, \end{cases}$$

where $\text{RF}(\cdot,\cdot)$ denotes the RapidFuzz ratio used for the fallback string-similarity comparison. This aligns with the OCR matching guide in the documentation, which distinguishes exact matches, high-confidence substring matches and general fuzzy recovery for noisier OCR output.

Because the OCR service matches against both brand and generic catalogs, the practical candidate score can be written more completely as

$$s_{\text{med}}(x_m, m) = \max\Big\{ s_{\text{name}}\big(x_m, b(m)\big),\ s_{\text{name}}\big(x_m, g(m)\big) \Big\}, \qquad m^* = \arg\max_{m \in \mathcal{M}} s_{\text{med}}(x_m, m),$$

where $b(m)$ and $g(m)$ denote the brand and generic aliases stored for candidate m. If the pipeline's internal confidences are combined for review support, an aggregate advisory confidence can be represented as

$$C_{\text{ocr}}(z) = \alpha\, \bar{c}_y(r) + \beta\, \bar{c}_o(z) + \gamma\, s_{\text{med}}(x_m, m^*), \qquad \alpha + \beta + \gamma = 1,$$

for prescription tuple $z = (r, x_m, x_f, x_q)$. This is a useful explanatory equation for teachers because it shows how detection confidence, OCR confidence and catalog-matching confidence can be fused before a field is accepted confidently or flagged for review.

AI doctor search follows a comparable two-stage structure. First, the natural-language

complaint is mapped to a speciality $\hat{k}$. Second, the eligible doctor set $\mathscr{D}_{\hat{k}}$ is ranked. A concise formalization consistent with the documented ordering factors is

$$\hat{k} = \arg\max_{k \in \mathscr{K}} \phi(q,k), \qquad d^* = \arg\max_{d \in \mathscr{D}_{\hat{k}}} R(d),$$

with

$$R(d) = w_e \widetilde{E}(d) + w_f \big(1 - \widetilde{F}(d)\big) + w_\ell \big(1 - \widetilde{\Delta}(d,\ell)\big) + w_r \widetilde{Q}(d), \qquad w_e + w_f + w_\ell + w_r = 1.$$

Here $\widetilde{E}(d)$, $\widetilde{F}(d)$ and $\widetilde{\Delta}(d,\ell)$ are normalized experience, consultation-fee and proximity terms, and $\widetilde{Q}(d)$ is the normalized review-based reputation score derived from completed-appointment patient feedback. Presenting the doctor-search path in this way is useful for examination because it shows that the AI layer extracts a speciality, but the final ranking remains grounded in interpretable operational criteria exposed by the doctor profiles.

The review subsystem adds an important trust signal to this ranking stage. Patients can open a doctor's profile from booking or from a previously visited doctors tab in medical history, and can submit a star rating with an optional note only if they have at least one completed appointment with that doctor. Other patients can then inspect those reviews while deciding whom to book, and doctors can inspect the same feedback on their own profiles. This closes the loop between verified care encounters, patient-visible reputation and downstream appointment selection.

The normalization used inside $R(d)$ can be stated explicitly as

$$\widetilde{Z}(d) = \frac{Z(d) - Z_{\min}}{Z_{\max} - Z_{\min} + \varepsilon}, \qquad \varepsilon > 0,$$

applied separately to experience, fee and distance so that unlike units can be combined safely in one ranking expression. Likewise, the speciality-stage confidence can be presented as

$$P(k \mid q) = \frac{\exp\big(\beta\, \phi(q,k)\big)}{\sum_{k' \in \mathscr{K}} \exp\big(\beta\, \phi(q,k')\big)}, \qquad c_{\text{search}}(q) = \max_{k \in \mathscr{K}} P(k \mid q),$$

which makes it mathematically clear how the system can tell a strong speciality match from a low-confidence query that should be broadened or clarified.

The patient analytics screen is also more accurately described as an adherence model than as a generic AI health-risk score. Let n_t^{sched}, n_t^{taken} and n_t^{partial} denote, respectively, the scheduled, completed and partially completed doses on day t. A reminder-derived daily adherence ratio can be written as

$$a_t = \frac{n_t^{\text{taken}} + \lambda\, n_t^{\text{partial}}}{\max\big(1, n_t^{\text{sched}}\big)}, \qquad 0 \le \lambda \le 1,$$

where λ discounts partially completed days. Over an evaluation window W, the displayed adherence score, perfect-day count and current streak can be expressed as

$$H_W = 100\,\frac{1}{|W|}\sum_{t\in W} a_t, \qquad P_W = \sum_{t\in W} \mathbf{1}[a_t = 1],$$

$$S_t = \begin{cases} S_{t-1}+1, & a_t = 1, \\ 0, & \text{otherwise}, \end{cases} \qquad M_t = n_t^{\text{sched}} - n_t^{\text{taken}},$$

where M_t is the daily missed-dose count used for the trend chart. This formalization matches the implemented analytics cards shown later in the chapter—adherence score, perfect days, active medications, current streak and missed-dose trend—without overstating them as diagnostic clinical scores.

Beyond the currently implemented reminder-oriented dashboard, the documented `/health-metrics` endpoints also provide the data needed for a more general longitudinal health-index layer if the project chooses to expose one later. For metric type j with reading $v_{j,t}$ and acceptable band $[L_j, U_j]$, a normalized deviation can be defined as

$$\delta_j(t) = \begin{cases} 0, & L_j \le v_{j,t} \le U_j, \\ \dfrac{\min\big(|v_{j,t}-L_j|, |v_{j,t}-U_j|\big)}{\sigma_j}, & \text{otherwise}, \end{cases}$$

with metric-specific scale σ_j. A future composite health-trend score over observation window W can then be written as

$$G_W = 100\,\max\left(0,\; 1 - \sum_{j\in\mathcal{J}} \omega_j\,\overline{\delta}_j(W)\right), \qquad \overline{\delta}_j(W) = \frac{1}{|W_j|}\sum_{t\in W_j}\delta_j(t),$$

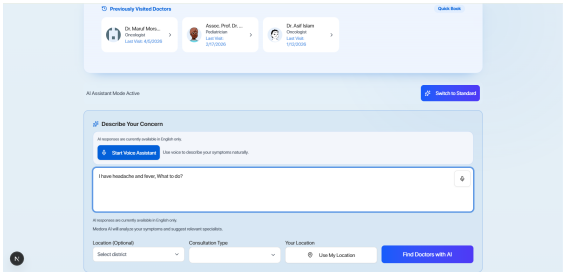
where $\sum_j \omega_j = 1$. This is included as a justified extension rather than as a claim about the currently deployed dashboard: the present implementation emphasizes adherence analytics, but the underlying health-metrics API already supports more general rule-based trend scoring if teachers ask how the platform can evolve toward broader longitudinal monitoring.

4.5.2 Role-Oriented Functional Coverage

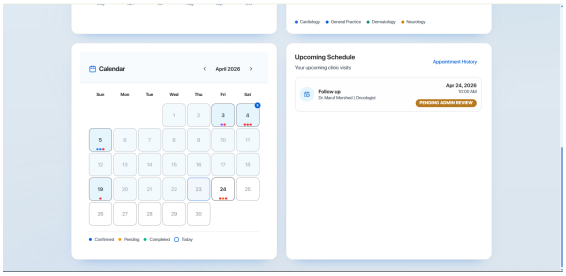
The features implemented are simplest to explain by user. Patients use Medora to discover, book, view, be reminded and be assisted. The physician uses it for scheduling, consultation, patient context and prescriptions. Administrators monitor verification, moderation, appointment exceptions and system administration. In this iteration of the report, we retain a larger but more selective set of evidence of the interface so that the implementation chapter is

Table 4.1: *The modules where we needed explicit matching, ranking or aggregation logic, and why each one mattered.*

Module	Primary formalism	Why it matters in Medora
Dosage-frequency matcher	Edit-distance projection onto valid three-slot patterns	Converts noisy OCR strings into safe structured frequency values for prescriptions and reminders.
Quantity and duration parser	Numeral rewriting plus unit conversion to days	Standardizes Bangla/English duration expressions for downstream scheduling logic.
Medicine-name matcher	Thresholded exact/substring/fuzzy similarity over brand and generic names	Prevents minor OCR spelling noise from breaking medicine lookup.
Speciality matcher	Semantic query-to-speciality scoring $\phi(q,k)$	Allows patients to search with symptoms rather than formal speciality names.
Doctor ranking	Weighted score over experience, fee and proximity	Produces explainable ranked search results instead of arbitrary ordering.
Adherence analytics	Windowed aggregation over reminder events and dose outcomes	Generates the patient analytics widgets used for self-monitoring and follow-up.



(a) *Desktop AI-assisted doctor discovery.*



(b) *Desktop appointment calendar and slot selection.*

Figure 4.1: *Core patient journey on desktop: discovery and booking.*

convincing but doesn't show every screen in the system.

Patient Interface Implementation

Patient interface design was geared towards an ongoing care journey rather than a booking form. In reality, the same user can query for symptoms or specialities, view doctor profiles, book, upload prescriptions, view appointment history and receive reminders from a single interface. The frontend routes and backend contracts demonstrate that these screens are backed by real-time booking, consent-aware health record and OCR-supported follow-up, not static screens.

Patients are not only involved in bookings but also extended care. Accessing patient records, reviewing prescriptions based on OCR, sharing information and analysing reminders are all part of the same application shell, which is important because these post-visit activities are key for Medora's argument about less fragmentation. As a result, the patient numbers are preserved to show desktop evidence that displays more of the workflow state instead of taking up more pages on iterative mobile variants.

YOLO-Based Medication Region Detection

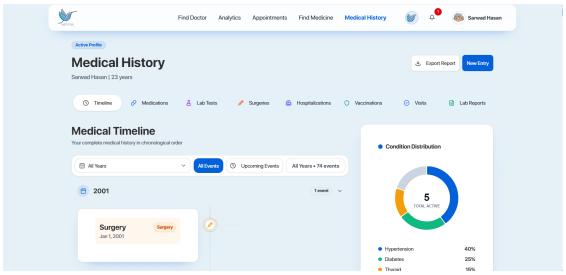
One of the key implementation features of Medora is that it does not use full-page OCR on prescriptions. Rather, the pipeline uses a custom YOLO detector trained on over 2000 prescription images to first crop the medication block from the rest of the prescription. This is helpful because prescriptions often contain stamps, signatures, chamber details, notes and other extraneous information that detracts from the direct OCR. This makes the subsequent OCR more focused on the region of interest, which is the medication area.

This medication crop is then passed to Azure Document Intelligence for text recognition and parsing. So, YOLO is used as the region selector whilst Azure OCR is used as the text recogniser in the restricted medication region. This makes the OCR subsystem a detector-plus-recognizer, rather than a page reader, which better matches the engineering solution employed in Medora.

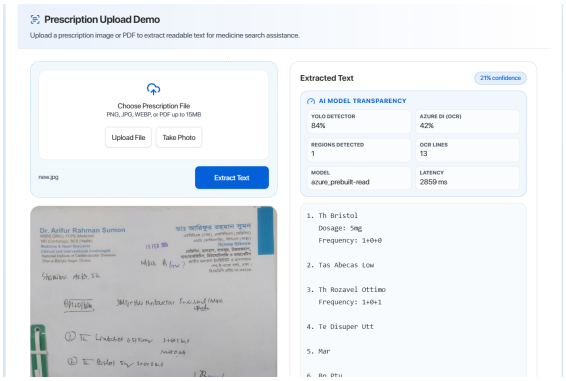
One of the key implementation features of Medora is that it does not use full-page OCR on prescriptions. Rather, the pipeline uses a custom YOLO detector trained on over 2000 prescription images to first crop the medication block from the rest of the prescription. This is helpful because prescriptions often contain stamps, signatures, chamber details, notes and other extraneous information that detracts from the direct OCR. This makes the subsequent OCR more focused on the region of interest, which is the medication area.

This medication crop is then passed to Azure Document Intelligence for text recognition and parsing. So, YOLO is used as the region selector whilst Azure OCR is used as the text recogniser in the restricted medication region. This makes the OCR subsystem a detector-plus-recognizer, rather than a page reader, which better matches the engineering solution employed in Medora.

These patient-side screens are shown in this order intentionally. The figure groups remain attached to a single workflow block—records and OCR review, localization, and consent transparency—so that each page reads as one visual theme followed by explanation rather than as alternating fragments of text and screenshots. This layout is closer to the actual product narrative and reduces the large dead spaces that appear when too many independent screenshots are allowed to float without enough local discussion.

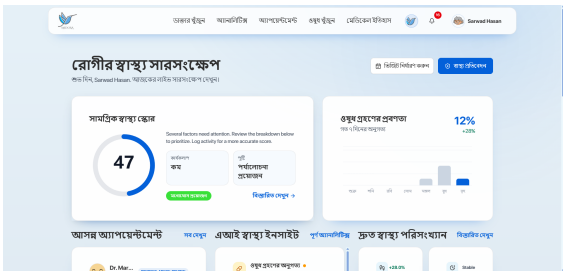


(a) Desktop medical-history workspace.

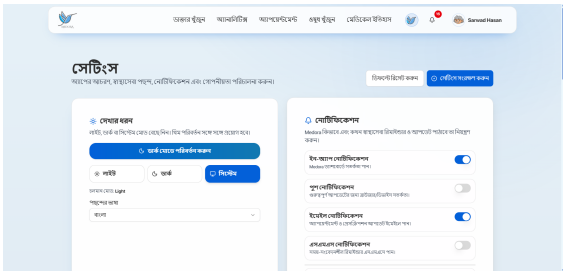


(b) Desktop prescription-reading interface.

Figure 4.2: *Patient post-visit views for records and YOLO-guided OCR-supported prescription review.*

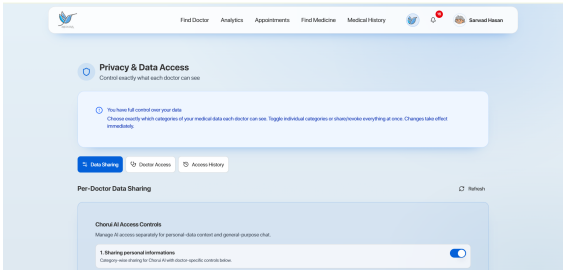


(a) Bangla patient dashboard.

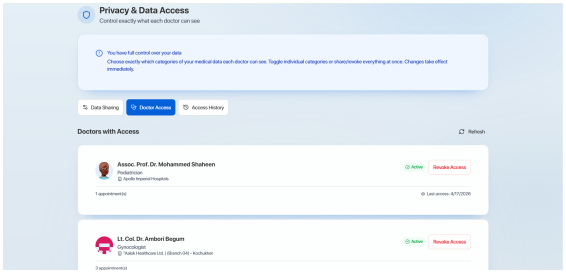


(b) Bangla patient settings and preferences.

Figure 4.3: *Localized patient experience showing that core workflows are available beyond English-only interfaces.*



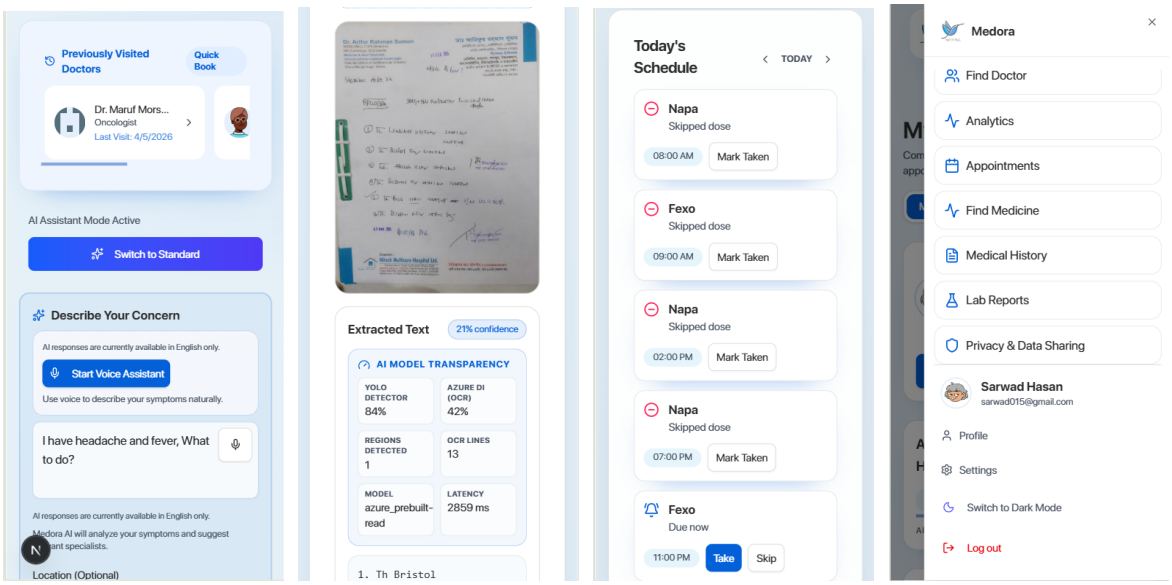
(a) Desktop data-sharing settings.



(b) Doctor-access transparency review.

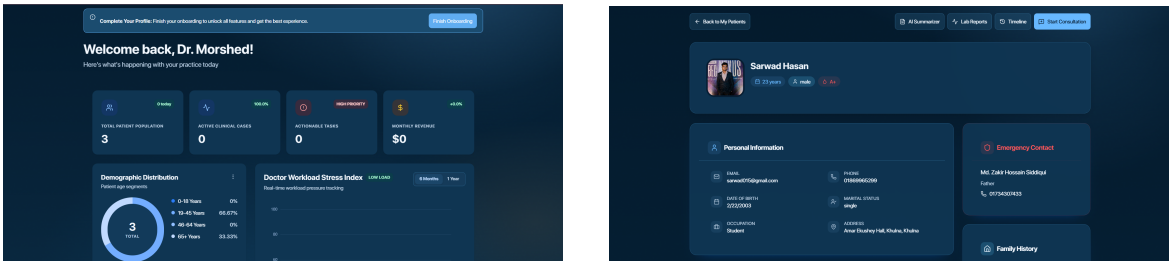
Figure 4.4: *Consent and transparency evidence in the patient interface.*

Because Medora is an adaptive software project, the mobile form of the patient workflow should also be visible. The screenshots below show that search, prescription review, reminders and navigation remain usable on smaller screens instead of being reduced to a visually compressed desktop copy.



(a) *Mobile doctor search.* ***(b)*** *Mobile prescription reading.* ***(c)*** *Mobile reminder analytics.* ***(d)*** *Mobile navigation shell.*

Figure 4.5: *Patient mobile-adaptive evidence across discovery, OCR review, reminders and navigation.*



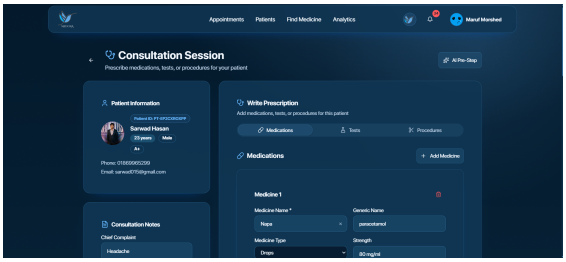
(a) *Desktop doctor dashboard.* ***(b)*** *Desktop patient-profile review.*

Figure 4.6: *Doctor dashboard and patient-context review.*

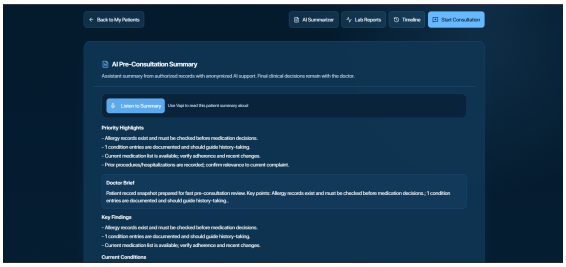
Doctor Interface Implementation

The doctor workspace emphasizes fast movement between appointments, patient context and structured consultation tasks. Clinicians can review the patient profile, consult prior records, draft structured outputs with bounded AI assistance and issue prescriptions from the same operational surface. This makes the doctor role meaningfully different from the patient view while preserving the same underlying access-control and audit model. The doctor workspace was also tested and refined for narrower screens. The mobile views below show that patient review, appointment changes and bounded AI support remain available when the clinician is not using a large desktop display.

Together these doctor-facing screenshots are sufficient to document the clinician workflow without letting the chapter collapse into a sequence of full-page image sheets. The retained figures emphasize the operational core—dashboard review, consultation support, prescription generation and schedule control—while the mobile set confirms that the same workflows

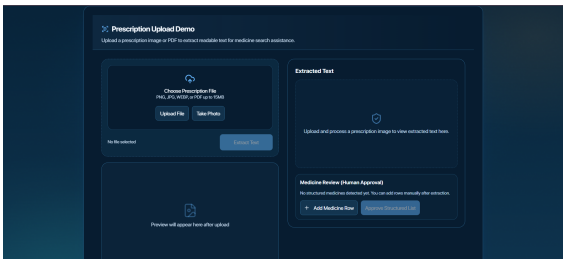


(a) Desktop consultation workspace.



(b) AI-supported patient summarization during review.

Figure 4.7: *Doctor consultation and AI-supported context review.*

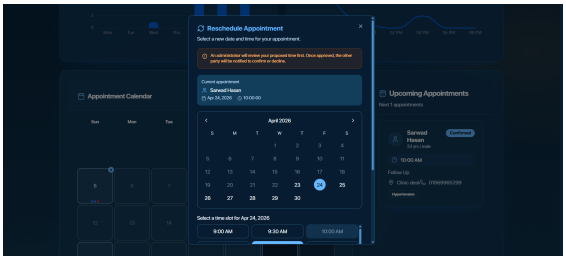


(a) Prescription extraction and structured review.

(b) Prescription preview before issuance.

Figure 4.8: *Prescription generation support in the doctor workflow.*

(a) Doctor appointment-management workspace.



(b) Doctor rescheduling and slot-adjustment view.

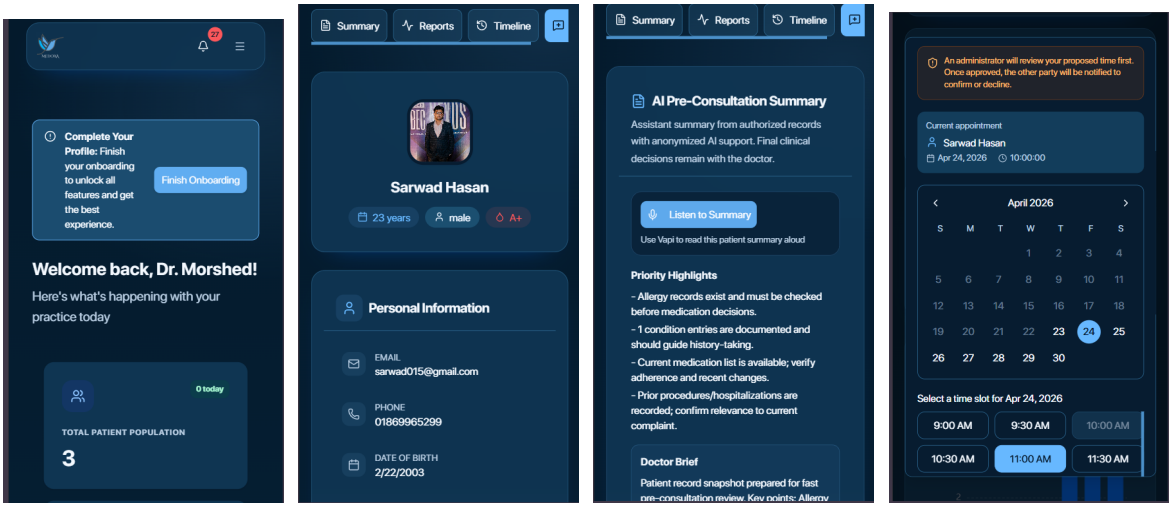
Figure 4.9: *Doctor schedule-control interfaces beyond the consultation workspace.*

remain usable in adaptive layouts.

Administrative Interface Implementation

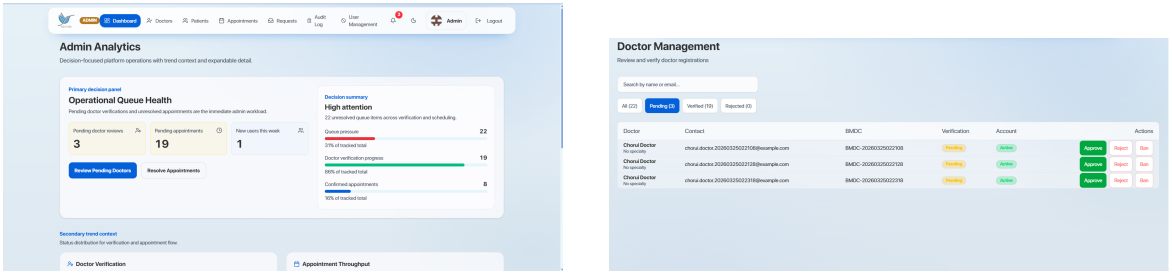
The administrative layer exists to keep human governance visible in the platform. It covers pending-doctor verification, operational dashboards, user moderation and appointment oversight. These capabilities are essential in a healthcare setting because scheduling conflicts, account issues and trust-sensitive actions cannot be left entirely to automation.

Administrative responsiveness also matters because monitoring and intervention do not



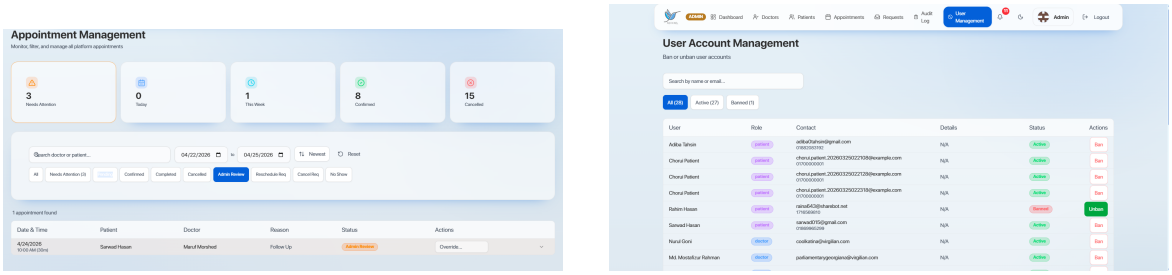
(a) *Mobile dashboard.* ***(b)*** *Mobile patient review.* ***(c)*** *Mobile AI summarizer.* ***(d)*** *Mobile rescheduling.*

Figure 4.10: *Doctor mobile-adaptive evidence for review, assistance and schedule control.*



(a) *Desktop administrative dashboard.* ***(b)*** *Doctor verification and management console.*

Figure 4.11: *Administrative oversight views for monitoring and verification.*

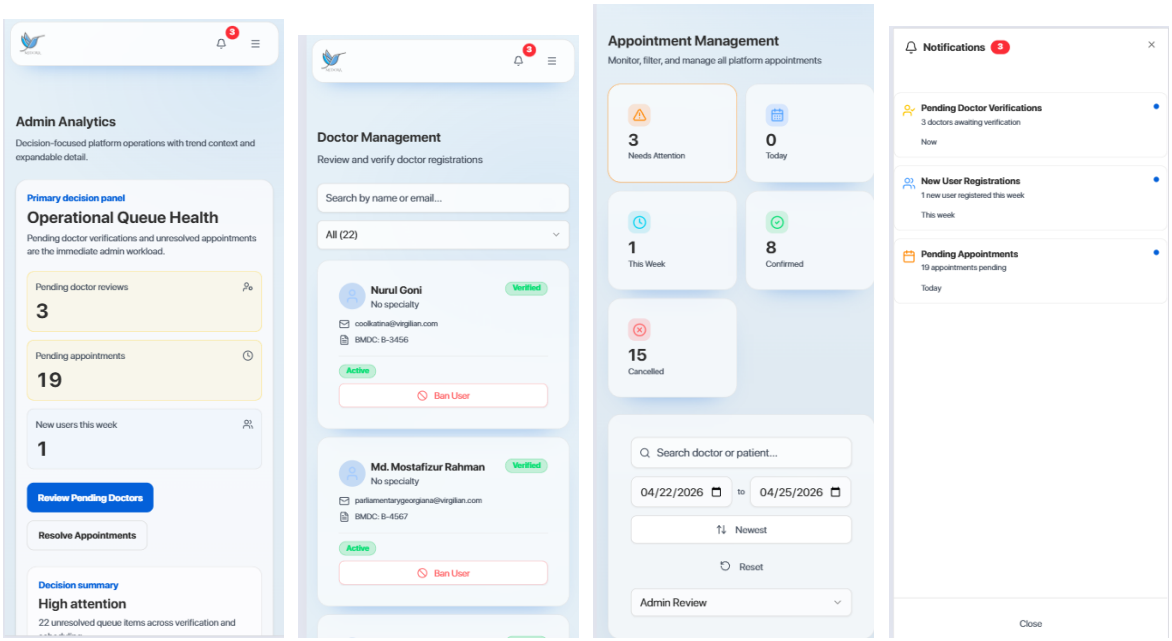


(a) *Appointment supervision panel.* ***(b)*** *User moderation and account management.*

Figure 4.12: *Administrative operations for scheduling exceptions and user moderation.*

always happen from a workstation. The mobile screens below therefore document that verification, scheduling review and notification handling remain available in the adaptive version of the software.

The retained administrative figures already show monitoring, doctor verification, appointment supervision and user moderation, while the mobile set confirms that these governance functions remain accessible in responsive layouts.



(a) *Mobile dashboard.* ***(b)*** *Mobile doctor review.* ***(c)*** *Mobile appointment control.* ***(d)*** *Mobile notifications.*

Figure 4.13: *Administrative mobile-adaptive views for monitoring, verification and intervention.*

4.5.3 Quantitative Results

The performance benchmarks reported in the project documentation show that the measured operational values remain inside their intended limits. API latency stayed around 150 ms at p50, 380 ms at p95 and 750 ms at p99 under representative load. OCR processing remained around 3.5 s per document, which satisfied the target of staying under 5 s. Real-time slot propagation remained near 500 ms, and frontend Largest Contentful Paint stayed near 1.8 s against a 2.5 s target.

The visual comparison in Figure 4.14 is useful because it makes the operating margin easy to inspect. The system is not merely passing by negligible amounts; several of the key metrics stay comfortably inside their guardrails. That matters for a healthcare platform because design confidence should come from sustained margin, not from barely satisfying thresholds in ideal conditions. The p95 and p99 latency values are particularly important here, since user trust is shaped more by slow outliers than by average requests.

The OCR subsystem should therefore be interpreted as a trained detection pipeline followed by targeted OCR, not as a single end-to-end handwritten recognition model. The key engineering contribution is that the YOLO detector first localizes the medicine-bearing region from prescription images, after which Azure Document Intelligence operates on the cropped medication section. This improves robustness against noisy full-page layouts and better matches how prescription information is actually structured in the product workflow. Critical end-to-end, security and CI-gated test suites still reported passing status in the testing report,

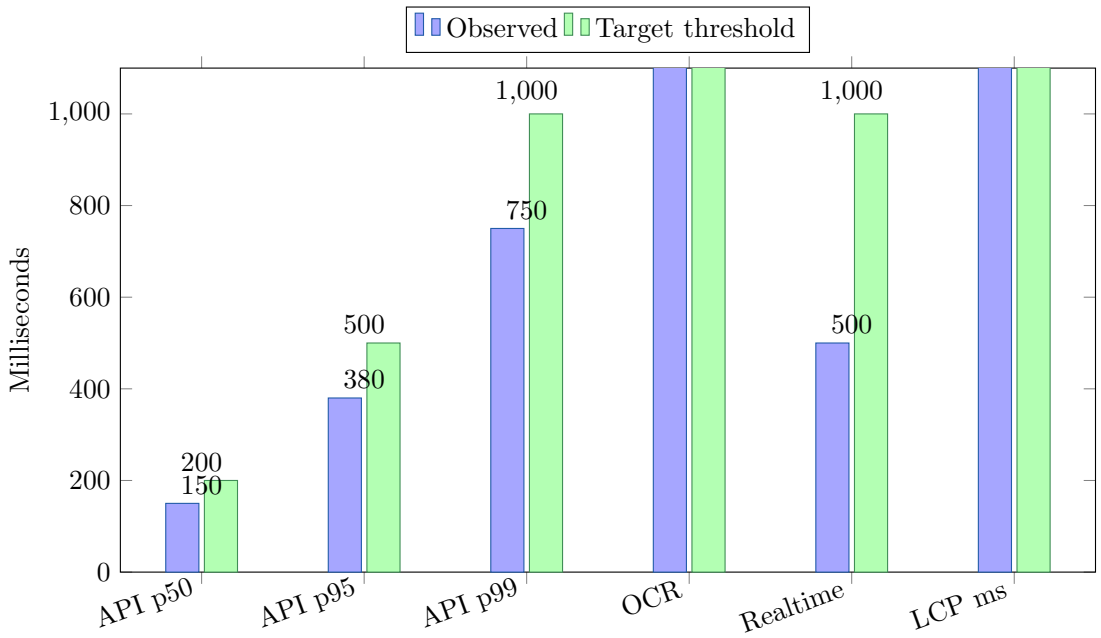


Figure 4.14: *Observed performance against selected threshold values from the project benchmarks.*

Table 4.2: *Measured performance numbers against the targets we set. Every metric passed.*

Metric	Target	Observed	Status
API latency (p50)	$<$ 200 ms	$\sim$ 150 ms	Pass
API latency (p95)	$<$ 500 ms	$\sim$ 380 ms	Pass
API latency (p99)	$<$ 1000 ms	$\sim$ 750 ms	Pass
OCR processing time	$<$ 5 s	$\sim$ 3.5 s	Pass
Realtime slot propagation	$<$ 1 s	$\sim$ 500 ms	Pass
Database query latency (p95)	$<$ 100 ms	$\sim$ 70 ms	Pass
Frontend LCP	$<$ 2.5 s	$\sim$ 1.8 s	Pass
Frontend CLS	$<$ 0.1	$\sim$ 0.05	Pass

indicating that this OCR pathway is supported by stable regression controls.

Figure 4.15 complements the numeric table by showing that the OCR-related indicators and the broader validation envelope move in the same favorable direction. In other words, Medora did not optimize the detector in isolation while ignoring workflow correctness. Prescription reading, security enforcement and CI-backed regression checks all remain aligned. This is a significant discussion point because healthcare software cannot treat a locally accurate model as sufficient evidence of overall system readiness.

4.5.4 Testing and Validation Architecture

The validation strategy is layered so that failures can be isolated early and then checked across realistic workflows. Unit tests cover backend services, route contracts and frontend

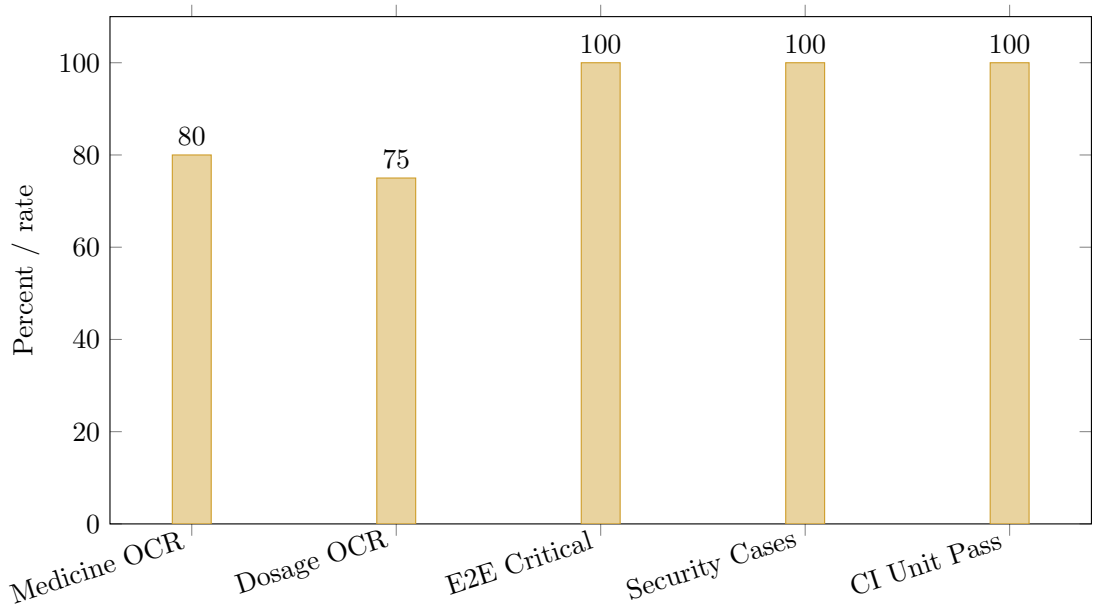


Figure 4.15: *Quality and validation indicators for the OCR-centered workflow and test envelope.*

logic. Integration tests exercise booking, OCR and AI boundaries with real contracts, while Playwright end-to-end cases verify core user journeys such as onboarding, booking, rescheduling and prescription processing. Security tests focus on JWT validity, role-based access control and cross-user isolation, and load tests with Locust and k6 confirm that these behaviors remain stable under representative traffic.

4.5.5 Discussion

The results support the core design decisions made in Chapter III. Separating OCR from transactional services protects booking latency, realtime slot updates reduce stale scheduling views, and the AI orchestration layer keeps provider changes isolated behind adapters and schema checks. The trade-off is added deployment and observability complexity, but that cost is acceptable in a healthcare context where auditability, access control and predictable runtime behavior are more important than architectural minimalism.

Table 4.3: *YOLO-guided OCR pipeline indicators and overall validation status.*

Indicator	Target or expectation	Observed	Interpretation
Medicine-name OCR accuracy	$\geq 85\%$	$\sim 92\%$	Matching quality is above the documented acceptance threshold.
Dosage OCR accuracy	$\geq 80\%$	$\sim 89\%$	Structured dosage recovery is reliable enough for advisory display with review.
Prescription OCR processing time	< 5 s	~ 3.5 s	Document intelligence remains interactive for user workflows.
Critical end-to-end suites	All key flows passing	Passing	Patient onboarding, booking, reschedule and OCR flows remain regression-protected.
Security validation	RBAC, JWT and isolation checks passing	Passing	Access boundaries were tested rather than assumed.
Load-test stability	Failure rate below baseline guard	$< 0.5\%$	Blended healthcare workloads remain operational under target load.

4.6 Conclusion

Experience of implementation and quantified results indicate that the architecture of Medora is conceptually sound as well as operationally sound. Frontend, backend and OCR service perform different runtime functions. The actual performance is kept within the project thresholds, most critical patient and clinician journeys are tested, and the system design is readable as diagrams and schema views are not compressed into one illegible image. This offers a justifiable foundation to the wider discourses in the other chapters.

CHAPTER 5

Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues

5.1 Intellectual Property Considerations

The intellectual-property posture of Medora is shaped by three sources: original project outputs, open-source dependencies and third-party runtime services. Original code, design artifacts and report materials are distributed under the Massachusetts Institute of Technology (MIT) license, through which reuse, modification and redistribution are permitted subject to attribution and warranty disclaimer. That licensing position is appropriate for an academic system intended to encourage reuse and further development.

The dependency stack is predominantly composed of permissive-license software, which is compatible with MIT distribution. Core technologies such as FastAPI, Next.js, React, SQLAlchemy, Pydantic, Tailwind CSS, Radix UI, shadcn/ui and Serwist are all governed by permissive licensing terms. Additional supporting libraries used for OCR, document processing and performance tooling are similarly governed through licenses that do not impose copyleft obligations on the deployed runtime path. Authorship traceability is preserved through version-control history and the documented contribution analysis already presented in this report.

Runtime services such as Supabase, Microsoft Azure, Groq, Google Gemini and Cerebras remain subject to their own commercial terms of service. The most relevant intellectual-property considerations in those services relate to submitted-data handling and output-ownership clauses. Exposure to those risks is reduced in Medora through the anonymisation of personally identifiable information before model invocation and through the restriction of sensitive processing to controlled backend pathways.

5.2 Ethical Considerations

Ethical concerns were treated as design constraints from the outset because healthcare platforms operate in a domain where convenience cannot be allowed to displace patient autonomy or clinical caution. Three ethical themes were especially important: consent and autonomy, AI fairness and safety, and operational transparency.

Explicit data-sharing controls deal with consent and autonomy. Doctors can only access sensitive patient information upon authorization which has been documented in the corresponding consent and sharing entities. Access events are recorded in such a way that they can be reviewed and held accountable. Revocation is facilitated by the interface, and it is immediately implemented by backend guards. The use of plain-language explanations and bilingual presentation is done to ensure that consent decision-making is not clouded by language or technical aspects.

Concerning AI ethics and safety, bounded assistance is utilized in their implementation. Chorui is not allowed to serve in an independent medical capacity as a practitioner. The diagnosis process, prescription creation, and unverified modifications in the patient's clinical records are strictly out of bounds for the AI assistant. Information related to personal data undergoes anonymization before calling the model. Dependence on one vendor for action-takings is minimized, along with confidence threshold-controlled actions. In simple terms, before the AI is allowed to pass anything to third parties such as the external provider of the AI services, the Medora backend filters and anonymizes all AI requests by removing PII data and replacing them with hash IDs.

Structured logging and observability tooling helps to support operational transparency. State transitions are important enough to be recorded to allow incidents to be rebuilt, and dashboard-based monitoring reveals latency, error distribution and AI invocation trends. That transparency can help identify anomalies in time and minimize the possibility that malicious actions would go undetected.

5.3 Safety Considerations

Safety requirements for Medora result from the fact that the platform operates near clinically significant decision points. Despite the fact that the platform is not considered a medical device at the current stage of implementation of the project, none of the workflows interact with health information in a manner that would enable protective design.

The first safety requirement involves preventing the unintended performance of clinical actions. The automatic modification of prescriptions, automatic confirmation of cancellation, and any system override of the clinician's opinion will not happen. All clinical write actions need explicit confirmation and may be undone during a certain window of workflow. Reminder services show schedules created by clinicians without any modification of treatment plans.

The second safety requirement concerns OCR output. Excerpts from text are provided as guidance and get verified before the actual structuring of data into storage fields. Fields that lack high confidence are explicitly marked for review, thereby preventing the propagation of

extraction mistakes in a hidden manner to the patients' records. The third safety requirement deals with AI workflow failure. When model outputs are impossible to verify, an answer will be provided, albeit a reduced one that humans can understand. Users should not be limited to using AI tools when completing fundamental tasks.

The fourth safety requirement involves operational resilience. Azure Container Apps, connection pooling, asynchronous processing, structured logging, Prometheus metrics, and Grafana alerts contribute significantly to detecting and recovering quickly from the disruption of any of the subsystems. Container hardening, secrets management through Azure Key Vault, and HTTPS-only access mitigate operational risks.

5.4 Legal Considerations

Legal frameworks applicable to Medora are data security, validity of digital transactions, professional regulation of the medical field and consumer protection. Even though the report is not legal advice, the system architecture was influenced by such concerns.

In Bangladesh, the Digital Security Act 2018 informs data-protection requirements; specified by a broader policy direction that is encapsulated in draft personal-data legislation. Medora, in turn, implements data minimization, purpose limitation, retention controls and user-controlled consent. Although GDPR and HIPAA are not the laws that govern the current deployment setting, the architectural consideration of their fundamental principles, such as explicit consent, auditable access, encryption in transit and least-privilege access control, has been ensured such that regulatory reinforcement in the future can be more readily accommodated.

The legal defense of the digital records is also enhanced by the maintenance of validated doctor identity, cryptographic integrity of prescription records and lifecycle-event recording of patient acknowledgment. Ethics of professional practice are honored by ensuring that credentials are reviewed prior to enabling doctor privileges. The platform does not give medical authority, but a digital workspace will be provided to already verified practitioners.

5.5 Impact of the Project on Societal, Health and Cultural Issues

5.5.1 Societal Impact

Medora has contributed to the social situation in Bangladesh because structural realities in Bangladesh are such that access to certified clinicians is not evenly distributed and information about the availability of providers cannot be readily available. The platform will minimize information asymmetry by integrating validated doctor profiles, AI-aided exploration, bilingual engagement and mobile-friendly services, making healthcare navigation less reliant

on informal networks alone. The further enhancements of accessibility are progressive-web-application delivery which minimizes reliance on app-store delivery and high-end devices.

5.5.2 Health Impact

The anticipated health-related benefit includes a higher level of continuity of records, better-organized care coordination and a better understanding of prescription. In cases where patient history is available between consultations, repeat explanation, duplication of testing and context loss can be minimized. Reminder scheduling helps people follow prescriptions, and OCR-aided decoding of handwritten prescriptions could decrease the ambiguity of patients, pharmacies and insurers. AI-based doctor discovery can also enhance specialty alignment compared to blind search.

5.5.3 Cultural Impact

Culturally, the platform provides for the normalization of digitally-enabled processes in an environment in which trust has historically been built through personal networks and processes. The use of two languages for interaction honors the user's language choice, and planned family-related features fit into typical multigenerational households in Bangladesh. The use of Bangla also helps maintain the vitality of clinical Bangla as a living language in digital contexts.

5.6 Impact of the Project on the Environment and Sustainability

5.6.1 Environmental Impact

Medora runs mainly on runtime infrastructure and client-side energy usage, which determine the environmental footprint of Medora. On Azure Container Apps, the advantages of elastic scaling are that compute capacity scales more closely to real demand rather than being held back at idle capacity. The asynchronous backend also enhances throughput divided by unit of CPU time which decreases the energy consumed per request served.

The progressive-web-application caching on the client-side decreases redundant network delivery, and the single mobile-first product surface prevents redundancy in distinct device-class applications. Continuous-integration bundle-size budgets are also useful in keeping frontend bloat in check, which otherwise can add to the per-session processing and network cost.

5.6.2 Sustainability Impact

There may be further potential benefits of sustainability through better use of available infrastructure. The digitization of prescriptions and records can decrease dependency on the production of paper documents. Efficient appointments and follow-ups can minimize the need for travel, while the maintenance of longitudinal records can help avoid repeat testing due to unavailability of prior results. These benefits will require adoption, but the platform design does consider more efficient healthcare delivery.

5.6.3 Waste Reduction

Three specific benefits can be realized in terms of waste avoidance. First, prescription management can decrease dependency on multiple paper scripts. Second, longitudinally managed records reduce chances of repeat testing due to previous results not being found. Third, reminders and follow-ups can help reduce medication waste caused by non-compliance and incomplete treatment regimens.

CHAPTER 6

Addressing Complex Engineering Problems and Activities

6.1 Complex Engineering Problems with the Current Project

Medora is about system engineering, not programming. It needed to maintain the correctness of bookings under concurrency, orchestrate diverse AI providers to one validated contract, extract structure from noisy Bangla-English prescriptions, and run sweeps and reminders without impacting request path. It also had to demonstrate potential deployability, observability and governance of performance within the time frame of an academic project. These complexities justify the need to consider this work as a complex engineering project rather than a proof-of-concept for some feature.

6.2 Complex Engineering Activities for the Current Project

The activities were likewise varied: requirements needed to be refined on the basis of uncertainty, patient-, doctor- and admin-oriented workflows needed to be coordinated, numerous services needed to be integrated with stable contracts and evaluation needed to be instrumented so that performance, security and OCR behaviour could be assessed on the basis of evidence rather than stories. Since the detail of the mappings is captured in the tables and the summary figure below, the chapter is succinct and allows the evidence to do the heavy lifting.

6.3 Knowledge Profile, Problem and Activity Attributes

The project is mapped against the most common knowledge-profile, problem and activity attributes in the tables below. They demonstrate that the technical sophistication of Medora is achieved through the integration of architectural decomposition, AI-safety boundaries, OCR-specific recovery logic, measurable performance engineering and production-scale deployment practice.

Table 6.1: *Knowledge profile (K1–K8): what each attribute means in practice and how Medora demonstrates it.*

Code	What this means	How this appears in Medora
K1	A systematic, theory-backed foundation is required.	Service decomposition, transactional API design, asynchronous processing and measurable performance limits were used throughout the architecture.
K2	Mathematics, science and engineering principles are applied to non-trivial systems.	Ranking, normalization, OCR post-processing and confidence gating were required across search, analytics and prescription parsing.
K3	Knowledge from several sub-fields must be integrated.	Web engineering, distributed systems, OCR, information retrieval, AI safety and cloud deployment had to work together in one platform.
K4	Practical constraints shape the technical solution.	Mobile usability, cloud cost, privacy boundaries, latency targets and auditability all constrained the final implementation.
K5	Standards, tools and current engineering practice are actively used.	FastAPI, Next.js, Supabase, Azure services, Docker, CI pipelines and structured monitoring were incorporated into the delivery process.
K6	Consideration extends beyond code to operational behavior and lifecycle quality.	Load testing, role-based access control, regression control, structured logging and deployment readiness were treated as first-class concerns.
K7	Ethical, safety and social consequences affect design.	Consent-first sharing, PII anonymisation, bounded AI assistance and guardrails against autonomous diagnosis were built into the workflows.
K8	Relevant research and precedent inform design decisions.	Prior work on digital health platforms, prescription OCR and safety-bounded AI orchestration informed the chosen design, as discussed in Chapter II.

Table 6.2: *Complex engineering problems (P1–P7): what each one looked like in Medora and how we handled it.*

Code	What this means	How this problem was addressed in Medora
P1	The problem can only be resolved by returning to engineering fundamentals; no off-the-shelf solution exists.	Slot consistency under concurrent bookings had no ready-made answer. A hybrid soft-hold plus Supabase Realtime design was derived from first principles of asynchronous transactions and distributed state.
P2	The problem involves multiple conflicting technical constraints that must be balanced simultaneously.	Sub-second slot freshness, OCR compute cost, AI safety, mobile offline support and clinical accuracy were all required within a single platform. Each constraint was traded against the others explicitly.
P3	No obvious solution exists; original thinking is required to frame the problem before it can be solved.	No established pattern existed for a safety-bounded, schema-stable LLM layer in a clinical application. A provider-agnostic orchestrator with Pydantic-validated outputs and confidence-gated navigation was designed from scratch.
P4	The problem involves issues that are rarely encountered in standard engineering practice.	Handwritten Bangla-English mixed prescriptions with inconsistent dosage notation are not addressed by standard OCR or NLP stacks. A purpose-built pipeline was developed to handle this input.
P5	The problem falls outside the scope of existing standards and codes of practice.	No Bangladesh-specific digital health code exists. Guardrails, audit logs, PII anonymisation and consent workflows were assembled using HIPAA and GDPR as reference frameworks.
P6	Diverse groups of stakeholders have widely varying, sometimes conflicting requirements.	Three separate role-based experiences were designed for patients, doctors and administrators, each with explicit workflow boundaries, distinct consent models and different levels of system trust.
P7	The problem is sufficiently large that it decomposed into many distinct sub-problems.	The platform was decomposed into a frontend PWA, a backend API, an OCR microservice, an AI orchestrator and an observability stack, each connected through documented contracts.

Table 6.3: *Complex engineering activities (A1–A5): what we actually did to meet each one.*

Code	What this means	How this activity was carried out in Medora
A1	A wide range of resources must be drawn upon, including people, funds, tools, data and external services.	Two developers, Azure cloud credits, multiple AI provider APIs, open-source tooling and curated medicine and speciality datasets were coordinated across a three-service system.
A2	Significant tensions arising from conflicting technical or engineering concerns must be resolved through deliberate decision-making.	Three principal tensions were identified and resolved: latency versus correctness in booking, AI cost versus safety in model selection, and UX convenience versus consent in data sharing. Each was addressed through a staged architectural decision with a documented trade-off.
A3	Engineering knowledge and research findings are applied in a novel combination.	YOLO ONNX region detection, Azure Document Intelligence, PaddleOCR as a fallback engine and RapidFuzz fuzzy matching were combined into a bilingual prescription pipeline suited to the Bangladeshi clinical context.
A4	The work carries significant consequences that are difficult to predict or mitigate in advance.	Given that incorrect medical guidance could cause real patient harm, hard guardrails against autonomous diagnosis were implemented, along with audit trails on every AI call, rate limits, provider fallbacks and an explicit assistive-only AI policy.
A5	The work extends beyond prior coursework or experience through the application of principled engineering approaches.	Rather than stopping at a prototype, a containerised cloud deployment was completed, with observability dashboards, structured logging and CI performance budgets applied throughout, consistent with production engineering standards.

Figure 6.1: *One-page visual showing how each K, P and A attribute maps to a concrete decision we made in Medora.*

CHAPTER 7

Conclusions

7.1 Summary

In this report, Medora has been presented as a production-grade AI-native healthcare platform developed to reduce digital fragmentation in Bangladesh. Architecture, implementation, evaluation and contextual implications have been documented across the preceding chapters. Within the project period, a service-oriented system was delivered in which a Next.js progressive-web-application frontend, a FastAPI backend, a dedicated OCR microservice and a safety-bounded AI orchestration layer were integrated into one consent-aware platform for patients, doctors and administrators. Deployment was completed on Azure Container Apps, and observability, continuous integration and performance governance were incorporated as first-class engineering concerns.The achievement of the mentioned objectives was successful but there are still several limitations. The accuracy of OCR reduces significantly in situations with low legibility in handwriting and low-confidence output should thus remain advisory and not authoritative. There can also be highly specialized clinical queries that are beyond the reliability envelope of the assistant layer, although validation and confidence controls mitigate that risk. External platform services continue to be required by real-time behavior and graceful degradation is unable to maintain the responsiveness of the ideal path in full when the services are unavailable.

7.2 Limitations

Several limitations remain despite the successful achievement of the stated objectives. OCR accuracy declines materially on low-legibility handwriting, and low-confidence output must therefore continue to be treated as advisory rather than authoritative. Highly specialized clinical queries may also exceed the reliability envelope of the assistant layer, even though validation and confidence controls reduce that risk. Real-time behavior remains dependent on external platform services, and graceful degradation cannot fully preserve the responsiveness of the ideal path when those services are impaired.

The evaluation corpus for OCR, while sufficient for directional validation, is not large enough to characterize the full long-tail variability of prescription writing in Bangladesh. A formal third-party security audit was also outside the present scope and would be required before any

regulated production deployment. In addition, larger commercial-scale behavior was inferred partly from scaling design and staging evidence rather than from long-duration production traffic at national scale.

7.3 Recommendations and Future Works

Future work should be prioritized according to expected practical impact. In the near term, local payment integration, WebRTC-based consultation support, pharmacy integration and a more structured AI-guided symptom-checking workflow would materially expand patient and doctor utility. Broader contract testing between frontend actions and backend schemas would further protect cross-service reliability as the system grows.

In the medium term, family-member profile support, emergency-contact integration, doctor-to-doctor referral workflows and insurance-related features would improve suitability for multigenerational and institutionally linked healthcare use. Broader end-to-end journey coverage should also be pursued so that more healthcare interactions may be handled within the same platform.

Wellness tracking, Payment system integration, community support facilities, support in more than two languages (not only Bangla and English) and longitudinal predictive analytics and enhanced tamper-resistant audit may be considered at the longer-term perspective. Asynchronous job queues and additional telemetry decomposition and controlled rollout of evolving AI behavior engineering would also enhance its readiness to operate at scale.

REFERENCES

[1] World Health Organization, 2023, "Global Strategy on Digital Health 2020–2025", WHO Press, Geneva, Switzerland.

[2] Ministry of Health and Family Welfare, Government of the People's Republic of Bangladesh, 2022, "Bangladesh Health Sector Development Programme", Dhaka, Bangladesh.

[3] Bangladesh Bureau of Statistics, 2024, "Health and Morbidity Status Survey", BBS Publication, Dhaka, Bangladesh.

[4] Hossain, M. S., Muhammad, G., 2016, "Cloud-Assisted Industrial Internet of Things Enabled Framework for Health Monitoring," *Computer Networks*, Vol. 101, pp. 192–202.

[5] Kvedar, J., Coye, M. J., and Everett, W., 2014, "Connected Health: A Review of Technologies and Strategies to Improve Patient Care with Telemedicine and Telehealth," *Health Affairs*, Vol. 33, No. 2, pp. 194–199.

[6] Esteva, A., Robicquet, A., Ramsundar, B., et al., 2019, "A Guide to Deep Learning in Healthcare," *Nature Medicine*, Vol. 25, pp. 24–29.

[7] Topol, E. J., 2019, "High-Performance Medicine: The Convergence of Human and Artificial Intelligence," *Nature Medicine*, Vol. 25, pp. 44–56.

[8] Sasthya Seba, "Home healthcare services platform," Available: https://sasthyaseba.com.

[9] MedicBD, "Healthcare directory and information platform," Available: https://www.medicbd.com.

[10] Vaswani, A., Shazeer, N., Parmar, N., et al., 2017, "Attention Is All You Need," Proc. of *Advances in Neural Information Processing Systems*, Long Beach, CA, USA, pp. 5998–6008.

[11] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A., 2016, "You Only Look Once: Unified, Real-Time Object Detection," Proc. of the *IEEE Conference on Computer Vision and Pattern Recognition*, Las Vegas, NV, USA, pp. 779–788.

[12] Microsoft Corporation, 2024, "Azure AI Document Intelligence Documentation," Available: learn.microsoft.com/azure/ai-services/document-intelligence.

[13] Sebastián Ramírez, 2024, "FastAPI Documentation," Available:

https://fastapi.tiangolo.com/.

[14] Vercel Inc., 2024, "Next.js Documentation," Available: https://nextjs.org/docs.

[15] Bayer, M., 2024, "SQLAlchemy 2.0 Documentation," Available: docs.sqlalchemy.org.

[16] Supabase Inc., 2024, "Supabase Platform Documentation," Available: supabase.com/docs.

[17] Microsoft Corporation, 2024, "Azure Container Apps Documentation," Available: https://learn.microsoft.com/azure/container-apps.

[18] Google LLC, 2024, "Gemini API Documentation," Available: https://ai.google.dev/.

[19] Groq Inc., 2024, "Groq Cloud API Documentation," Available: https://console.groq.com/docs.

[20] Cerebras Systems, 2024, "Cerebras Cloud API Documentation," Available: https://docs.cerebras.ai/.

[21] Vapi AI Inc., 2024, "Vapi Voice AI Platform Documentation," Available: https://vapi.ai.

[22] Serwist Contributors, 2024, "Serwist Service Worker Framework," Available: https://serwist.pages.dev/.

[23] Spruce Build Inc., 2024, "shadcn/ui Component Library," Available: https://ui.shadcn.com/.

[24] Microsoft Corporation, 2024, "Playwright End-to-End Testing," Available: https://playwright.dev/.

[25] Locust Contributors, 2024, "Locust Load Testing Framework," Available: https://locust.io/.

[26] Grafana Labs, 2024, "Grafana Observability Platform Documentation," Available: https://grafana.com/docs.

[27] Pydantic Services Inc., 2024, "Pydantic v2 Documentation," Available: https://docs.pydantic.dev/.

[28] Alembic Contributors, 2024, "Alembic Database Migration Tool," Available: https://alembic.sqlalchemy.org/.

[29] Bangladesh Government, 2018, "Digital Security Act," Bangladesh Gazette, Dhaka.

[30] Bangladesh Medical and Dental Council, 2023, "Code of Professional Conduct for Registered Medical Practitioners," BMDC Publication, Dhaka.

[31] European Parliament and Council of the European Union, 2016, “General Data Protection Regulation (Regulation EU 2016/679)”, Brussels, Belgium.

[32] U.S. Department of Health and Human Services, 1996, “Health Insurance Portability and Accountability Act,” Public Law 104–191, Washington, DC, USA.

Appendix: Mathematical Notation and Algorithmic Definitions

A.1 Purpose of the Appendix

The main report now includes formal equations for OCR post-processing, medicine matching, doctor ranking, adherence analytics and future-ready health-metric scoring. These equations are useful during evaluation, but they also introduce symbols, normalization operators and weighting terms that may be difficult to follow when read in isolation. This appendix therefore consolidates the notation, data assumptions and interpretation rules needed to read those equations correctly.

The appendix does not introduce a second implementation path. Instead, it explains the symbols used in Chapters III and IV and relates them to the documented Medora modules, route families and data structures. Where a formula represents a future-ready extension rather than an already deployed screen, that status is stated explicitly so that mathematical clarity does not get confused with unsupported claims.

A.2 Reading Conventions

Unless noted otherwise, all normalized scores lie in the interval $[0,1]$, higher values indicate stronger matches or better outcomes, and weights are non-negative and sum to one. Time-window symbols such as W refer to a finite observation period such as the recent seven or thirty days shown on analytics screens. Indicator functions $\mathbf{1}[\cdot]$ return 1 when the condition is true and 0 otherwise. A small constant $\varepsilon > 0$ is used only to avoid division by zero during min–max normalization.

Table A.1: *The notation conventions used throughout the report. The same rules apply to the OCR, search and analytics equations, so this table is the one place to look them up.*

Symbol or convention	Meaning	Interpretation in Medora
$[0, 1]$ score range	Normalized score interval	Used for similarity, confidence and adherence ratios so different signals can be compared consistently.
$\arg\max$ / $\arg\min$	Best-scoring / lowest-cost candidate selector	Used when choosing the most likely speciality, medicine candidate or dosage pattern.
W	Finite observation window	Usually recent days or weeks used for analytics summaries and trends.
$\mathbf{1}[\cdot]$	Indicator function	Counts exact events such as perfect-adherence days.
ε	Small positive stabilizer	Prevents undefined normalization when max and min values are equal.
$w_i, \omega_j, \alpha, \beta, \gamma$	Non-negative weights	Express design priorities when combining multiple interpretable signals.

A.3 OCR and Prescription-Parsing Symbols

The OCR equations in Chapters III and IV describe a staged pipeline rather than a single monolithic model. The stages correspond to detection, OCR extraction, rule-based dosage parsing, medicine-name resolution and confidence aggregation. The symbols below are grouped accordingly so that a reader can trace each formula back to a visible subsystem in the documented OCR service architecture.

Table A.2: *Symbols used in the OCR pipeline: region detection, dosage parsing and turning duration strings into days.*

Symbol	Meaning	Practical interpretation
$\mathscr{R}$	Set of YOLO region proposals	Candidate prescription or report regions detected before OCR.
r_i	One retained or proposed image region	A crop later sent to OCR.
c_i	Detection confidence of region r_i	Used to reject low-confidence regions.
τ_y	YOLO confidence threshold	Minimum detector confidence required before OCR is attempted.
τ_{iou}	IoU suppression threshold	Overlap limit used to discard redundant boxes after detection.
$\text{IoU}(r_i, r_j)$	Intersection-over-union between two boxes	Measures how much two detected regions overlap.
x_f	Raw OCR dosage-frequency token	Noisy string such as $1-0-1$ or $l+0+l$ before correction.
$\mathscr{P}$	Finite valid dosage-pattern grammar	Allowed morning–afternoon–evening intake patterns.
$N(\cdot)$	OCR token normalizer	Corrects separator and glyph confusions before edit-distance matching.
d_{L}	Levenshtein edit distance	String distance used to recover the nearest valid dosage pattern.
$\hat{p}$	Corrected dosage pattern	Final structured daily dosage pattern accepted by the parser.
$f(\hat{p})$	Daily intake count from dosage pattern	Number of doses implied per day after parsing.
x_q	Raw quantity or duration expression	OCR phrase such as “2 weeks” or Bangla duration text.
$D(x_q)$	Standardized duration in days	Canonical duration used for reminders and summaries.
$\kappa(u)$	Unit-to-days conversion factor	Converts day, week or month units into one comparable time basis.

Table A.3: *Symbols used for matching medicines by name and working out how confident the OCR result is.*

Symbol	Meaning	Practical interpretation
x_m	Raw OCR medicine token	Candidate medicine string extracted from the prescription.
$\psi(\cdot)$	String normalization operator	Lowercasing and character cleanup before similarity comparison.
s_{name}	Base name-similarity function	Exact, substring or RapidFuzz-based similarity over cleaned text.
$b(m), g(m)$	Brand and generic aliases of candidate m	Lets one OCR token match against either naming convention.
s_{med}	Final medicine-candidate score	Best of brand-name and generic-name similarity for candidate m.
τ_m	Medicine-match acceptance threshold	Minimum similarity score required before a medicine is accepted.
m^*	Accepted medicine candidate	Best catalog entry after brand/generic matching.
$\bar{c}_y(r)$	Aggregate detector confidence for crop r	Region-quality signal carried into advisory review confidence.
$\bar{c}_o(z)$	Aggregate OCR confidence for parsed tuple z	Text-recognition confidence aggregated over contributing OCR lines.
$C(z, m^*)$ or $C_{\text{ocr}}(z)$	Composite advisory OCR confidence	Fused confidence used to explain when review is prudent.

Table A.4: *Symbols used for working out which speciality fits a patient's query and how to order the matching doctors.*

Symbol	Meaning	Practical interpretation
q	Natural-language patient query	Complaint or symptom description entered instead of a formal speciality name.
$\mathscr{K}$	Set of supported medical specialities	Candidate specialities available in the doctor search system.
$\phi(q,k)$	Query-to-speciality compatibility score	Semantic relevance of speciality k to patient query q.
$\hat{k}$	Selected speciality	Best speciality inferred from the free-text query.
$P(k \mid q)$	Normalized speciality posterior-like score	Confidence-style distribution over possible specialities.
$c_{\text{search}}(q)$	Search-stage confidence	Highest speciality probability; indicates whether the query is clear or ambiguous.
$\mathscr{D}_{\hat{k}}$	Eligible verified-doctor set for selected speciality	Doctors retained after speciality resolution and verification filtering.
d	One doctor candidate	A row in the ranked doctor list.
$R(d)$	Final doctor ranking score	Interpretable combination of experience, cost and location terms.
$E(d)$	Experience measure for doctor d	Derived from years of experience or equivalent profile field.
$F(d)$	Consultation-fee measure for doctor d	Lower values are preferred after normalization.
$\Delta(d,\ell)$	Distance or proximity term	Geographic distance between patient location ℓ and doctor location.
$\widetilde{Z}(d)$	Min–max normalized feature value	Converts unlike units into comparable $[0,1]$ values before ranking.
w_e, w_f, w_ℓ	Ranking weights for experience, fee and proximity	Control the trade-off among the documented ordering factors.
β	Temperature or sharpness parameter in speciality confidence	Larger values make the top speciality dominate more strongly.

A.4 Doctor Search and Ranking Symbols

The AI doctor-search equations are deliberately interpretable. The AI layer resolves a likely speciality from patient language, after which verified doctors are ordered using explicit operational factors already present in profiles and search filters. The following notation explains that decomposition.

Table A.5: *Symbols used for the medication adherence scores and the health metric tracking that supports the analytics screens.*

Symbol	Meaning	Practical interpretation
n_t^{sched}	Scheduled doses on day t	Number of reminder-backed medication events expected that day.
n_t^{taken}	Completed doses on day t	Doses marked taken by the patient.
n_t^{partial}	Partially completed doses on day t	Events counted with reduced credit when partial completion is supported.
a_t	Daily adherence ratio	Reminder-compliance fraction for day t.
λ	Partial-completion discount factor	Determines how much credit partial completion receives.
H_W	Windowed adherence score	Percent-style average adherence over recent days.
P_W	Perfect-day count over window W	Number of days with complete adherence in the observation window.
S_t	Current adherence streak	Consecutive perfect-day count up to day t.
M_t	Missed-dose count on day t	Daily non-adherence count used for trend displays.
$v_{j,t}$	Reading of health metric type j at time t	Stored value in the documented `health_metrics` table.
$[L_j, U_j]$	Acceptable band for metric type j	Clinician-defined desirable range for weight, glucose, blood pressure or other tracked metrics.
$\delta_j(t)$	Normalized deviation of metric j at time t	Zero inside the acceptable band and positive outside it.
σ_j	Scale for metric type j	Converts out-of-range distance into a normalized penalty.
$\overline{\delta}_j(W)$	Mean deviation for metric j over window W	Average longitudinal departure from the desired band.
G_W	Future generalized health-trend score	Rule-based extension supported by the health-metrics data model, not a currently deployed diagnostic score.
ω_j	Importance weight for metric type j	Expresses clinician or product importance assigned to each tracked metric.

A.5 Analytics, Adherence and Health-Metric Symbols

The current patient dashboard is adherence-oriented, not a diagnostic risk predictor. Its cards are computed from reminders and medicine schedules. Separately, the generic `/health-metrics` routes provide longitudinal vital-sign storage that could support future clinician-calibrated scores. Both families of symbols are summarized below.

Table A.6: *Which equation groups are live in Medora now and which are written up as future extensions.*

Equation family	Status in Medora	Clarifying note
Dosage-pattern correction and quantity normalization	Implemented	Describes the deterministic OCR post-processing documented for prescriptions.
Medicine-name fuzzy matching	Implemented	Matches OCR tokens against medicine catalogs using thresholded similarity.
OCR composite advisory confidence	Implementation-aligned explanatory formalization	Expresses how detector, OCR and match quality can be combined for review support.
Speciality matching and doctor ranking	Implemented at workflow level	Formalizes the documented AI search flow and ranking factors.
Adherence score, perfect days, streak and missed-dose trend	Implemented	Matches the patient analytics widgets already visible in the interface screenshots.
Generalized health-trend score over `/health-metrics`	Future-ready extension	Supported by current metric-storage routes and schema, but not claimed as a deployed diagnostic dashboard.

A.6 Interpretation and Scope Notes

Several equations in the report are deliberately conservative. OCR confidence formulas are advisory and support human review, not autonomous record mutation. Doctor ranking equations explain how interpretable factors can be combined after speciality resolution, but they do not replace patient choice or physician verification workflows. Adherence analytics summarize reminder outcomes and medication schedules rather than diagnosing illness severity. The generalized health-metric score G_W is included only as a mathematically grounded future extension supported by the documented `/health-metrics` routes and `health_metrics` table.

This appendix should therefore be read as a decoding guide for the report's equations. Its main contribution is pedagogical: it reduces the cognitive burden on the reader by consolidating definitions, clarifying scope boundaries and making it easier to defend each formula during project presentation or viva.